

Computational State Primitives, Universal Data Pathing, and Session Management

The Runtime Layer for VDR-LLM-Prolog

Registry: [\[HOWL-VDR-8-2026\]](#)

Series Path: [\[HOWL-VDR-1-2026\]](#) → [\[HOWL-VDR-2-2026\]](#) → [\[HOWL-MATH-3-2026\]](#) → [\[HOWL-MATH-4-2026\]](#) → [\[HOWL-VDR-3-2026\]](#) → [\[HOWL-VDR-4-2026\]](#) → [\[HOWL-LLM-1-2026\]](#) → [\[HOWL-VDR-5-2026\]](#) → [\[HOWL-VDR-6-2026\]](#) → [\[HOWL-VDR-7-2026\]](#) → [\[HOWL-VDR-8-2026\]](#)

DOI: 10.5281/zenodo.20215703

Date: May 2026

Domain: Applied Philosophy / Systems Architecture / Runtime State Management

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.6.

Abstract

VDR-5 specified the knowledge architecture for an exact-arithmetic language model: scoped knowledge bases, constraints, provenance, and first-class data surfacing. VDR-6 specified the execution layer: 255 computational primitives, command tokens, operational environments, and credential gating. VDR-7 specified the complete lifecycle: training, feedback, evaluation, deployment, monitoring, and retirement as KB operations. All three papers describe systems that store knowledge and execute operations. None of them address the LLM's own runtime working memory, the addressing system that connects the growing KB tree, or the ability to capture and restore the live state of a session.

This paper specifies three tightly integrated capabilities. Runtime data primitives — LRU caches, counters, locks, queues, stacks, ring buffers, and bitsets — give the LLM bounded, named, scoped working memory inside the KB struct, accessible through command tokens. Universal dotted-path addressing with integer ID acceleration gives every KB, every data primitive, and every fact a structured namespace with $O(1)$ runtime access. Session snapshots, cloning, and disposable operational clones give the system the ability to capture complete live state atomically, fork sessions for experimentation, and maintain stability through controlled recycling of drift-prone clones.

The paper also specifies how command tokens use dotted-path references to minimize the LLM's generative burden — command construction becomes reference selection from a known vocabulary, not freeform text generation.

The three capabilities depend on each other. Data primitives create state worth snapshotting. Dotted paths give snapshots efficient references to that state. Session management

provides the lifecycle for data primitives that would otherwise accumulate drift indefinitely. Together they complete the runtime layer that sits between the knowledge architecture (VDR-5) and the execution layer (VDR-6).

1. Context: The VDR-LLM-Prolog System

For readers entering the series at this paper, the VDR-LLM-Prolog system is a language model architecture with three foundational properties.

Exact arithmetic. Every number in the system is an exact fraction — an integer numerator over an integer denominator. There are no floating-point numbers. There is no rounding. Adding $1/7$ and $1/13$ produces exactly $20/91$, not a float near 0.2198. This was established in VDR-1 through VDR-4 [HOWL-VDR-4-2026] and verified by 705 tests across 23 mathematical domains with zero computation errors. The machine learning stack — softmax, autodiff, transformer forward and backward passes — operates entirely in exact fractions.

Knowledge bases as universal containers. Everything in the system is stored in Knowledge Bases (KBs). A KB is a structured collection of facts (what is true), rules (what follows from what), and constraints (what must hold). KBs organize in a tree where child KBs inherit from parents. The active topic determines which KBs are in scope — switching topics activates relevant KBs and deactivates others without deleting anything. User accounts are KBs. The organizational hierarchy is a KB tree. Constraints live inside the KBs they govern. This was specified in VDR-5 [HOWL-VDR-5-2026].

Primitives and command tokens. The LLM does not compute by token prediction. It invokes deterministic primitives through structured command tokens in its output stream. 211 pure primitives (sorting, arithmetic, string operations, linear algebra, graph algorithms) provide guaranteed-correct computation. 44 operational primitives (file I/O, compilation, script execution, networking) interact with sandboxed environments under positive credential grants. This was specified in VDR-6 [HOWL-VDR-6-2026].

Lifecycle as KB operations. The complete model lifecycle — data sourcing, corpus preparation, tokenization, training, fine-tuning, human feedback, evaluation, deployment, monitoring, updates, retirement — is specified as KB operations. Every phase produces queryable KBs. The entire lineage from raw data to retired model is one tree. This was specified in VDR-7 [HOWL-VDR-7-2026].

VDR-8 addresses what is missing from this stack: the LLM's own runtime working memory, the addressing system that navigates the KB tree at scale, and the ability to manage session-level state.

2. The Problem

2.1 No Structured Working Memory

The LLM currently has two places for working state. The scratchpad is a per-turn computation space — it holds intermediate results during a single response and is pruned afterward. Working data sets are scoped key-value bindings on topic KBs — they hold named values persistently.

Neither provides the lightweight operational state that programs routinely use. There is no bounded cache of recent items. There is no counter that tracks how many times something has happened. There is no flag that indicates an operation is in progress. There is no ordered queue of pending work. There is no undo stack. Without these, the LLM must scan conversation history or re-query KB facts to reconstruct state that a simple data structure would maintain naturally.

The scratchpad is also a single undifferentiated space. The LLM cannot maintain separate channels for separate concerns — recent errors in one cache, recent user questions in another, recent test results in a third. Everything goes into one buffer.

2.2 No Structured Addressing

KBs identify themselves by name strings. As the KB tree grows — VDR-7 specifies approximately 60 KB types across 12 lifecycle phases — navigation by flat names becomes unwieldy. The name `source_wikipedia_2026_03` carries no structural information about where the KB sits in the tree, what its parent is, or how to reach it from elsewhere. Cross-branch references require knowing the exact flat name of a distant KB. There is no hierarchical namespace, no relative addressing, and no fast access path for runtime operations that traverse the tree frequently.

2.3 No Session State Management

The system accumulates live state across a session — scratchpad contents, active scope configuration, working data bindings. This state is valuable but fragile. There is no way to capture it atomically, restore it exactly, or fork it into parallel experiments. If a session drifts into a degraded state — accumulated context pollution, stale working memory, compounding edge cases — the only option is to start over and lose all accumulated context.

2.4 These Problems Interact

The absence of data primitives means there is nothing worth snapshotting beyond raw text. The absence of structured addressing means snapshots cannot efficiently reference which KBs to capture. The absence of session management means data primitives, once created, accumulate drift with no recovery mechanism. The three solutions must be designed together because they depend on each other.

3. Runtime Data Primitives

3.1 The Principle

Standard computer science data structures — LRU caches, counters, locks, queues, stacks, ring buffers, bitsets — become fields on the KB struct, accessible through command tokens, scoped and inherited through the KB tree like facts, rules, and constraints.

These are not a separate system. They follow the same architectural principle established in the VDR-5 addendum, where constraints moved from a separate registry into the KBs they govern. Data primitives belong to the KB they serve.

3.2 LRU Caches

A named, bounded, least-recently-used cache. The LLM creates a channel for a specific concern — recent errors, recent user questions, recent test results — with a declared capacity. When the cache is full, the oldest entry is evicted automatically. Multiple named channels keep separate concerns in separate caches.

```
LRU = struct {  
  
  name: Text,  
  
  capacity: i32,  
  
  entries: []LRUEntry,  
  
  created_at: i32,  
  
};  
  
LRUEntry = struct {  
  
  key: Text,  
  
  value: Term,  
  
  last_accessed: i32,  
  
};  
Operations:
```

Operation	Signature	Description
lru create	(name, capacity) → void	Create named LRU on active KB
lru push	(name, key, value) → void	Insert or update entry, evict oldest if full
lru get	(name, key) → ?Term	Lookup by key, updates access time
lru peek	(name, count) → []Entry	Most recent N entries without updating access
lru contains	(name, key) → bool	Membership test
lru size	(name) → i32	Current entry count
lru clear	(name) → void	Remove all entries
lru evict	(name, key) → void	Remove specific entry

3.3 Counters

Named exact integer counters. Increment, decrement, read, reset. Track retry attempts, iteration counts, event occurrences without re-querying and counting KB facts every turn.

```
Counter = struct {
  name: Text,
  value: i32,
  min_value: i32,          // floor (default 0)
  max_value: i32,          // ceiling (default max_i32)
  created_at: i32,
};
```

Operations:

Operation	Signature	Description
counter create	(name, ?min, ?max) → void	Create named counter on active KB
counter inc	(name) → i32	Increment by 1, return new value
counter dec	(name) → i32	Decrement by 1, return new value
counter add	(name, delta) → i32	Add delta (positive or negative), return new

Operation	Signature	Description
counter get	(name) → i32	Read current value
counter reset	(name) → void	Reset to min value
counter set	(name, value) → void	Set to specific value within bounds

Counters clamp at their bounds. Incrementing past max_value stays at max_value. Decrementing past min_value stays at min_value.

3.4 Locks

Named non-blocking flags. Acquire sets a flag. Check reads it. Release clears it. Nothing blocks — the LLM’s turn-based execution model cannot support blocking operations. Locks are coordination signals: the LLM checks a lock before starting an operation that should not run concurrently, or reacts when it observes a lock has been released.

```
LockState = struct {
    name: Text,
    held: bool,
    held_by: ?Text,          // identifier of holder
    acquired_at: ?i32,
    notes: ?Text,
};
```

Operations:

Operation	Signature	Description
lock create	(name) → void	Create named lock on active KB
lock acquire	(name, ?holder, ?notes) → bool	Set flag, return true if was free
lock release	(name) → void	Clear flag
lock check	(name) → bool	Return true if held
lock holder	(name) → ?Text	Return holder identifier
lock force release	(name) → void	Release regardless of holder (admin)

3.5 Queues

Named bounded FIFO queues. Push items to the back, pop from the front. Manage pending work, buffered requests, ordered task lists.

```
BoundedQueue = struct {  
  
  name: Text,  
  
  capacity: i32,  
  
  items: []Term,  
  
  created_at: i32,  
  
};
```

Operations:

Operation	Signature	Description
queue create	(name, capacity) → void	Create named queue on active KB
queue push	(name, item) → bool	Push to back, return false if full
queue pop	(name) → ?Term	Remove and return front item
queue peek	(name) → ?Term	Read front item without removing
queue size	(name) → i32	Current item count
queue is empty	(name) → bool	Empty test
queue is full	(name) → bool	Full test
queue clear	(name) → void	Remove all items
queue to list	(name) → []Term	Read all items as list without removing

3.6 Stacks

Named bounded LIFO stacks. Push and pop. Undo history, nested context tracking, depth-first work management.

```
BoundedStack = struct {  
  
  name: Text,  
  
  capacity: i32,
```

```

items: []Term,

created_at: i32,

};

```

Operations:

Operation	Signature	Description
stack create	(name, capacity) → void	Create named stack on active KB
stack push	(name, item) → bool	Push to top, return false if full
stack pop	(name) → ?Term	Remove and return top item
stack peek	(name) → ?Term	Read top item without removing
stack size	(name) → i32	Current item count
stack is empty	(name) → bool	Empty test
stack clear	(name) → void	Remove all items
stack to list	(name) → []Term	Read all items top-to-bottom

3.7 Ring Buffers

Named fixed-size circular buffers. Writing to a full buffer overwrites the oldest entry. Rolling summaries, sliding windows, recent-N tracking without explicit eviction management.

```

RingBuffer = struct {

name: Text,

capacity: i32,

items: []Term,

write_pos: i32,

count: i32,

created_at: i32,

};

```

Operations:

Operation	Signature	Description
ring create	(name, capacity) → void	Create named ring buffer on active KB
ring write	(name, item) → void	Write item, overwrite oldest if full
ring read all	(name) → []Term	All items in chronological order
ring read last	(name, count) → []Term	Most recent N items
ring size	(name) → i32	Current item count (up to capacity)
ring clear	(name) → void	Reset to empty

3.8 Bitsets

Named fixed-size bit arrays. Track completion of numbered items, feature flags, membership in small fixed sets.

```

Bitset = struct {
  name: Text,
  width: i32,
  bits: []bool,
  created_at: i32,
};

```

Operations:

Operation	Signature	Description
bitset create	(name, width) → void	Create named bitset on active KB
bitset set	(name, index) → void	Set bit at index
bitset clear	(name, index) → void	Clear bit at index
bitset test	(name, index) → bool	Test bit at index
bitset count	(name) → i32	Count of set bits
bitset all set	(name) → bool	All bits set
bitset any set	(name) → bool	Any bit set
bitset reset	(name) → void	Clear all bits
bitset to list	(name) → []i32	Indices of all set bits

3.9 Growth Limits and Lifecycle Classification

Every data primitive has a declared maximum capacity at creation. LRU capacity, queue max size, stack max depth, ring buffer size, bitset width — all bounded. No unbounded growth. The limits are small by design. Tracking 50 recent items per named channel is more than sufficient for most purposes while remaining cheap to maintain and fast to snapshot.

Data primitives are classified as **live state**, not persistent knowledge.

Classification	Contents	Survives Reset	Captured by Snapshot	Examples
Persistent	Facts, rules, constraints, connections	Yes	No (always present)	KB facts, Prolog rules
Live	Data primitives, scratch-pad, working data, active scope	No	Yes	Counters, queues, LRUs

A session reset clears all live state to defaults — zero counters, empty queues, released locks, empty caches — while persistent knowledge remains untouched. A session snapshot captures all live state for later restoration. This distinction is central to the session management system in Section 5.

3.10 Mutation Logging

All data primitive mutations are logged as provenance facts:

```
Fact: primitive_mutation(  
  kb_path("root.project.vdr.training"),  
  primitive_type("counter"),  
  primitive_name("retry_count"),  
  operation("inc"),  
  old_value(2),
```

```
new_value(3),  
turn(47),  
source("llm_scratchpad")).
```

The owner can query the mutation history of any data primitive through the surfacing layer.

4. Universal Dotted-Path Addressing

4.1 The Principle

Every KB has a dotted path that encodes its position in the tree. The path is the human-readable address. An integer ID provides $O(1)$ runtime access. Both representations coexist — the LLM and users work with paths, the runtime works with integers.

4.2 Path Structure

The dotted path follows the KB tree hierarchy, with each dot-separated segment naming one level:

```
root  
root.programming  
root.programming.zig  
root.programming.zig.networking  
root.programming.c  
root.programming.c.networking  
root.project  
root.project.vdr  
root.project.vdr.training  
root.project.vdr.training.run_01  
root.stories
```

```

root.stories.london

root.stories.london.characters

root.sessions

root.sessions.stable_operator_v1

```

Parent-child relationships are encoded in the path. `root.programming.zig` is a child of `root.programming` by construction. No separate parent field is needed to determine ancestry — the path contains it.

4.3 Path Rules

Rule	Description	Example
Root	Every path begins with <code>root</code>	<code>root</code>
Separator	Dot separates levels	<code>root.project.vdr</code>
Segment characters	Lowercase alphanumeric and underscore	<code>root.project</code> <code>vdr.training</code> <code>run 01</code>
No trailing dot	Paths do not end with a dot	<code>root.project not</code> <code>root.project.</code>
Unique	No two KBs share a path	Enforced by path registry
Depth limit	Maximum path depth is configurable	Default 16 levels
Segment length	Maximum segment length is configurable	Default 64 characters

4.4 Integer ID Registry

```

PathRegistry = struct {
    path_to_id: HashMap(Text, i32),
    id_to_path: []Text,
    next_id: i32,
};

```

When a KB is created, it is assigned the next available integer ID. The mapping is permanent for the lifetime of the system — a path always resolves to the same integer ID. If a KB is deleted, its ID is retired, not reused.

```

"root"                                → 0

```

"root.programming"	→ 1
"root.programming.zig"	→ 2
"root.programming.zig.networking"	→ 3
"root.programming.c"	→ 4
"root.programming.c.networking"	→ 5
"root.project"	→ 6
"root.project.vdr"	→ 7
"root.project.vdr.training"	→ 8
"root.project.vdr.training.run_01"	→ 9

4.5 Runtime Integer Access

All internal operations use integer IDs. Scope chains become integer arrays:

```
// Active scope for root.project.vdr.training.run_01
scope: [9, 8, 7, 6, 0]
```

Walking the scope is iterating a short array of i32 values. No string parsing, no dot splitting, no hash lookups per level.

Connections reference integer targets:

```
// Connection stored internally
connection(target_id: 5, relationship: "references", ...)
```

Data primitive access resolves to two integers — KB ID and slot ID:

```
// counter_get("root.project.vdr.training.run_01.retry_count")
// resolves at runtime to:
counters[9][slot_3]
```

4.6 Data Primitive Slot IDs

Data primitive names within a KB can also receive integer slot IDs, making the full access path two i32 values:

```
SlotRegistry = struct {  
    kb_id: i32,  
    name_to_slot: HashMap(Text, i32),  
    slot_to_name: []Text,  
    next_slot: i32,  
};
```

When a data primitive is created, it receives a slot ID within its KB:

```
// Within KB 9 (root.project.vdr.training.run_01):  
"retry_count"    → slot 0  
"recent_errors" → slot 1  
"training_lock"  → slot 2  
"pending_fixes"  → slot 3
```

Full access: `counters[kb_9][slot_0]` — two integer lookups.

4.7 Command Token Path Resolution

The LLM emits dotted paths in command tokens because paths are readable and part of the root KB vocabulary. The command token executor resolves the path to an integer ID once at the beginning of execution, then all downstream operations use the integer.

```
// LLM emits:  
CMD: counter_inc("root.project.vdr.training.run_01.retry_count")  
  
// Executor resolves:  
kb_id = path_registry.resolve("root.project.vdr.training.run_01") → 9  
slot_id = slot_registry[9].resolve("retry_count") → 0
```

```
// Runtime executes:
```

```
counters[9][0] += 1
```

The path resolution happens once per command token. If the same KB is referenced multiple times in one turn, the resolved integer ID is cached for the duration of the turn.

4.8 Mounting

A KB from one branch of the tree can appear at a path in another branch through mounting:

```
CMD: kb_mount("root.project.vdr.refs.c_net",
              source: "root.programming.c.networking",
              mode: read_only)
```

Now `root.project.vdr.refs.c_net` is a mount point that resolves to KB ID 5 (`root.programming.c.networking`). The VDR project can reference C networking knowledge through its own subtree without cross-branch queries.

```
Mount = struct {
  mount_path: Text,
  mount_id: i32,
  source_path: Text,
  source_id: i32,
  mode: enum { read_only, read_write, snapshot, mirror },
  created_at: i32,
  created_by: Text,
};
```

Mode	Reads	Writes	Updates From Source	Use Case
read only	Yes	No	Live (sees source changes)	Reference external knowledge

Mode	Reads	Writes	Updates From Source	Use Case
read write	Yes	Yes (requires grant)	Live	Collaborative editing
snapshot	Yes	No	Frozen at mount time	Stable reference point
mirror	Yes	No	Live, automatic	Synchronized read-only copy

Mount cycle detection: before creating a mount, the system traces the source's mount chain. If the `mount_path` appears anywhere in that chain, the mount is rejected. This prevents A mounting B and B mounting A.

4.9 Connections as KB Field

Each KB holds its own connections list as a field on the struct. Connections are typed, directional links to other KBs.

```

Connection = struct {
  target_id: i32,
  target_path: Text,           // for display and export
  relationship: Text,
  direction: enum { inbound, outbound },
  phase: ?Text,               // lifecycle phase if applicable
  created_at: i32,
  notes: ?Text,
};

```

Connections travel with the KB on export. The topology is distributed across the KBs that own it, following the same principle as constraints (VDR-5 addendum): the thing being described holds its own description.

Connection examples:

```
KB: root.project.vdr.corpus_v1
```



```

connections:

connection(target: root.project.vdr.sources.wikipedia,

           relationship: "sourced_from", direction: inbound)

connection(target: root.project.vdr.tokenized_v1,

           relationship: "tokenized_into", direction: outbound)

```

The VDR-6 graph primitives operate directly on the collected connections across KBs, using integer IDs as node identifiers. Topology queries — reachability, shortest path, connected components — are standard graph operations on the connection graph.

5. Compact Command Token Construction

5.1 The Problem With Generated Commands

In current LLM function-calling systems, the model generates structured JSON or similar syntax by token prediction. Each character of the function name, each argument key, each argument value is a predicted token. A complex function call might require generating 50-100 tokens of structured syntax. Every token is a prediction that can go wrong — misspelled function names, wrong argument types, malformed structure, hallucinated parameters.

5.2 Reference Selection, Not Text Generation

In VDR-LLM-Prolog, command token construction is fundamentally different. The 255 primitives from VDR-6, plus the data primitive operations from this paper, are facts in the root KB — always in scope, always accessible. The LLM does not generate a function name character by character. It selects a reference from a known, finite vocabulary.

The command token the LLM emits is compact:

```
CMD: counter_inc(root.project.vdr.training.run_01.retry_count)
```

This is three references:

1. **Primitive reference:** `counter_inc` — a known name from the root KB primitive registry.
2. **KB path reference:** `root.project.vdr.training.run_01` — a dotted path in the KB tree.
3. **Slot reference:** `retry_count` — a named data primitive within that KB.

None of these are generated text in the creative sense. The primitive name is a fixed vocabulary item. The KB path is a structural address the LLM has seen in scope queries.

The slot name is a label the LLM itself created (or queried from the KB). The LLM's task is selection and assembly of known references, not generation of novel syntax.

5.3 Why This Is More Reliable

Token prediction reliability depends on how constrained the output space is. Generating a novel English sentence is a high-entropy task — many tokens are plausible at each position. Selecting from 300 known primitive names and pointing at known KB paths is a low-entropy task — the correct output is heavily determined by the intent and the available references.

The root KB always contains:

```
KB: root.primitives

  facts:

primitive("counter_inc", category: "counter", args: ["name"], returns: "i32")
primitive("lru_push", category: "lru", args: ["name", "key", "value"], returns: "i32")
primitive("queue_pop", category: "queue", args: ["name"], returns: "?Term")
primitive("lock_check", category: "lock", args: ["name"], returns: "bool")
...

// All 300 primitives with signatures
```

Because these are exact KB facts — stored as exact integer-weighted entries, not fuzzy patterns in continuous weight space — the LLM's access to the primitive vocabulary is precise. The root KB is always in scope. Its contents do not degrade through attention layers or context window pressure.

5.4 Arguments Are Addresses, Not Values

For operations on data that already exists in the KB, the command token does not pass the data by value through the token stream. It passes a reference — a dotted path to where the data lives.

```
// NOT this (value in token stream):

CMD: list_sort([47, 3, 91, 15, 82, 66, 29, 44, 8, 73], ascending)

// THIS (reference to KB location):
```

```
CMD: list_sort (root.project.vdr.gyms.unsorted_results, ascending)
```

The data stays in the KB. The command token says where to find it. The primitive reads directly from the KB using integer ID access. The LLM never needs to serialize a 50-element list into its output stream just to sort it.

For small literal arguments — a capacity of 20, a counter delta of 1, a lock name — the value is inline in the token. The rule is: if the value is a short literal, inline it. If the value is stored data, reference it by path.

5.5 Command Token Structure Revised

With dotted-path addressing, the command token structure from VDR-6 is refined:

```
CommandToken = struct {  
  
  type: CommandType,  
  
  primitive_id: i32,           // resolved from root.primitives  
  
  args: []CommandArg,  
  
  store_result: ?Text,        // dotted path to store result  
  
  await: bool,  
  
};  
  
CommandArg = union {  
  
  path_ref: Text,             // dotted path to KB data  
  
  literal_i32: i32,  
  
  literal_text: Text,  
  
  literal_bool: bool,  
  
  literal_fraction: (i32, i32), // numerator, denominator  
  
};
```

The executor resolves all path references to integer IDs before dispatching to the primitive. The primitive operates on integer-addressed data. The result is stored at the path specified by `store_result`, or returned to the scratchpad if no store location is specified.

5.6 Full Command Token Example

A concrete example showing the complete flow:

```
// LLM scratchpad:

CMD: kb_query(root.project.vdr.gyms, latest_gym_number) → 24

CMD: counter_inc(root.project.vdr.gyms.gym_count) → 25

CMD: lock_acquire(root.project.vdr.training_active, holder: "gym_creation")

CMD: lru_push(root.project.vdr.recent_actions, "creating_gym_25", "tensor_a

// LLM visible output:

TEXT: "Writing gym 25 for tensor algebra."

CMD: ENV_UPLOAD(env_vdr, <script>, root.project.vdr.gyms.gym_25_script)

CMD: ENV_EXEC(env_vdr, "python3", root.project.vdr.gyms.gym_25_script)

CMD: STORE_RESULT(task_049, root.project.vdr.gyms.gym_25_result)

TEXT: "Tests running. Task: task_049."
```

Every CMD line is a fixed primitive name plus dotted-path references. The LLM selected `counter_inc` from the primitive vocabulary. It pointed at `root.project.vdr.gyms.gym_count` — a path it knows because it's in scope. The entire command token is three references assembled together. The generative burden is minimal.

6. Session Management

6.1 The Principle

The complete live state of a session — active scope, scratchpad, all data primitives across all active KBs — is capturable as one atomic snapshot and restorable exactly. Sessions can be forked into independent clones. Clones can be recycled when drift is detected.

6.2 Session Snapshot Structure

```
SessionSnapshot = struct {
```

```

id: i32,
name: Text,
path: Text,                // stored under root.sessions.*
created_at: timestamp,
created_by: Text,
// Scope state
active_scope: []i32,        // KB IDs in scope
active_topic: i32,          // current topic KB ID
secondary_scopes: []i32,    // explicitly activated secondary KBs
// Scratchpad state
scratchpad: RingBuffer,
// All live KB states
kb_states: HashMap(i32, KBLiveState),
// Session metadata
turn_count: i32,
total_commands_issued: i32,
notes: ?Text,
};

KBLiveState = struct {
kb_id: i32,
counters: HashMap(i32, i32),
locks: HashMap(i32, LockState),
lrus: HashMap(i32, LRU),

```

```

queues: HashMap(i32, BoundedQueue),

stacks: HashMap(i32, BoundedStack),

buffers: HashMap(i32, RingBuffer),

bitsets: HashMap(i32, Bitset),

working_data: ?WorkingDataSet,

};

```

6.3 Core Operations

Operation	Signature	Description
session snapshot	(name, ?notes) → void	Capture all live state atomically
session restore	(name) → void	Restore all live state from snapshot
session reset	() → void	Clear all live state to defaults
session clone	(source name, clone name) → void	Create independent session from snapshot
session kill	(name) → void	Destroy a cloned session's live state
session list	() → []Text	List all saved snapshots
session diff	(name a, name b) → Diff	Structured comparison of two snapshots
session info	(name) → SessionInfo	Metadata about a snapshot

6.4 Snapshot Captures Live State Only

The snapshot captures data primitives, scratchpad, working data, and scope configuration. It does not capture persistent KB facts, rules, constraints, or connections — those are always present in the KB tree and do not need to be duplicated.

This means a snapshot is small. It contains counter values, queue contents, LRU entries, lock states, bitset bits, ring buffer contents, working data bindings, and scope pointers. It does not contain the knowledge — only the working memory.

Restoring a snapshot puts the live state back exactly: every counter at its captured value, every queue in its captured order, every lock in its captured state, every LRU with its captured entries. The persistent KBs underneath may have changed since the snapshot was taken — facts may have been added or retracted by other sessions or system operations. The snapshot restores the working memory, not the knowledge.

6.5 Session Reset

Reset clears all live state across all in-scope KBs:

- All counters reset to their `min_value`
- All locks released
- All queues emptied
- All stacks emptied
- All LRU caches emptied
- All ring buffers emptied
- All bitsets cleared
- Scratchpad cleared
- Working data optionally cleared or retained (configurable)

Persistent KB contents are untouched. The session starts fresh with the same knowledge but no working memory.

6.6 Session Cloning

Cloning creates an independent copy of a snapshot's live state. The clone references the same persistent KBs but has its own counters, queues, locks, caches, and scratchpad.

```
CMD: session_snapshot("stable_v1")
```

```
CMD: session_clone("stable_v1", "experiment_a")
```

```
CMD: session_clone("stable_v1", "experiment_b")
```

Experiment A and experiment B start from identical live state. They diverge as each session issues different commands. Neither session's live state modifications are visible to the other.

Persistent KB modifications are visible across sessions. If experiment A asserts a new fact into `root.project.vdr.results`, experiment B sees it on its next KB query. The isolation boundary is live state, not persistent knowledge.

6.7 Disposable Operational Clones

The snapshot and clone mechanisms enable a stability pattern: build a session to a verified stable state, snapshot it, run disposable clones off the snapshot, and recycle them when drift is detected.

The workflow:

Phase 1: Build stable operator

- Work a session interactively
- Get all the right KBs active, constraints loaded, data primitives configured
- Verify the session is in a known-good state
- CMD: `session_snapshot("stable_operator_v1")`

Phase 2: Run disposable clones

- CMD: `session_clone("stable_operator_v1", "worker_001")`
- Worker handles requests for N turns
- Worker commits results to persistent KBs
- Drift constraint fires → CMD: `session_kill("worker_001")`
- CMD: `session_clone("stable_operator_v1", "worker_002")`
- Repeat

6.8 Drift Detection Constraints

Drift constraints live in the KBs that the clone operates in. They use the same constraint system from VDR-5:

```
constraint("max_session_turns", operational, active,  
condition(session_turns < 200),  
on_violation("kill_and_reclone"))  
  
constraint("context_saturation", operational, active,  
condition(context_usage < fraction(9, 10)),  
on_violation("kill_and_reclone"))  
  
constraint("denominator_drift", operational, active,  
condition(max_working_denominator < 2^48),
```



```

on_violation("kill_and_reclone"))

constraint("error_rate_drift", operational, active,

condition(counter_get("errors") < counter_get("requests") * fraction(1, 20))

on_violation("kill_and_reclone"))

```

When any drift constraint fires, the clone's live state is destroyed and a fresh clone launches from the same snapshot. The snapshot never degrades — it is frozen state. The clones are disposable workers. The snapshot is the factory.

6.9 Clone Work Preservation

When a clone is killed, only its live state is destroyed. Any facts, rules, or constraints the clone asserted into persistent KBs survive. The work product persists. The working memory is discarded.

This means a clone can:

1. Receive a request
2. Compute results using data primitives and scratchpad
3. Assert the results into a persistent KB
4. Be killed

The next clone sees the committed results because they are in the shared persistent layer. The killed clone loses only its counters, queues, caches, and other ephemeral state — the state that was going to drift anyway.

6.10 Stable Operator Versioning

The stable operator snapshot can itself be versioned:

```

CMD: session_snapshot("stable_operator_v1")

// ... improve the operator, add better constraints, configure more data primi

CMD: session_snapshot("stable_operator_v2")

// ... v2 develops a problem ...

CMD: session_clone("stable_operator_v1", "worker_fallback")

```

The stable operator lineage mirrors the model checkpoint lineage from VDR-7. Each version is a verified, frozen, cloneable baseline. Roll back to an earlier operator version if a later one has problems.

6.11 Session Storage

Snapshots are stored as KBs under the session management path:

```
root.sessions
root.sessions.stable_operator_v1
root.sessions.stable_operator_v2
root.sessions.experiment_a
root.sessions.experiment_b
```

They are KBs. They have dotted paths. They have integer IDs. They are queryable, versionable, constrainable, and addressable through all the same mechanisms as every other KB in the system.

7. Revised KB Struct

With all VDR-8 additions integrated:

```
KnowledgeBase = struct {
  // Identity
  name: Text,
  path: Text,                      // dotted path
  id: i32,                          // integer ID from path registry
  // Persistent knowledge (VDR-5)
  facts: FactSet,
  rules: RuleSet,
  // Constraints (VDR-5 addendum)
  constraints: []Constraint,
  // Connections (VDR-8)
```

```

connections: []Connection,

// Working data (VDR-5)
working_data: ?WorkingDataSet,

// Runtime data primitives (VDR-8)
lrus: HashMap(Text, LRU),
counters: HashMap(Text, Counter),
locks: HashMap(Text, LockState),
queues: HashMap(Text, BoundedQueue),
stacks: HashMap(Text, BoundedStack),
buffers: HashMap(Text, RingBuffer),
bitsets: HashMap(Text, Bitset),

// Tree structure
parent_id: ?i32,
children_ids: []i32,
mounts: []Mount,                                // VDR-8

// Metadata
visibility: enum { public, internal, owner_only },
frozen: bool,
owner: ?Text,
created_at: i32,
last_modified: i32,
};

```

Every field that was a text name reference in earlier specifications (parent, children) is now available as both a dotted path (for humans and export) and an integer ID (for runtime).

The struct carries both representations.

8. Primitive Additions

8.1 New Primitive Categories

VDR-8 adds three primitive categories to the VDR-6 set.

Category 15 (renumbered from VDR-6 operational): Data Primitive Operations

#	Primitive	Type	Description
1	lru create	Pure	Create named LRU cache
2	lru push	Pure	Insert or update LRU entry
3	lru get	Pure	Lookup by key
4	lru peek	Pure	Read recent N entries
5	lru contains	Pure	Key membership test
6	lru size	Pure	Current entry count
7	lru clear	Pure	Remove all entries
8	lru evict	Pure	Remove specific entry
9	counter create	Pure	Create named counter
10	counter inc	Pure	Increment by 1
11	counter dec	Pure	Decrement by 1
12	counter add	Pure	Add delta
13	counter get	Pure	Read value
14	counter reset	Pure	Reset to min value
15	counter set	Pure	Set to specific value
16	lock create	Pure	Create named lock
17	lock acquire	Pure	Set lock flag
18	lock release	Pure	Clear lock flag
19	lock check	Pure	Read lock state
20	lock holder	Pure	Read lock holder
21	lock force release	Pure	Admin release
22	queue create	Pure	Create named queue
23	queue push	Pure	Push to back
24	queue pop	Pure	Pop from front
25	queue peek	Pure	Read front item
26	queue size	Pure	Current count
27	queue is empty	Pure	Empty test
28	queue is full	Pure	Full test
29	queue clear	Pure	Remove all items
30	queue to list	Pure	Read all as list
31	stack create	Pure	Create named stack
32	stack push	Pure	Push to top
33	stack pop	Pure	Pop from top
34	stack peek	Pure	Read top item

#	Primitive	Type	Description
35	stack size	Pure	Current count
36	stack is empty	Pure	Empty test
37	stack clear	Pure	Remove all items
38	stack to list	Pure	Read all top-to-bottom
39	ring create	Pure	Create named ring buffer
40	ring write	Pure	Write item (overwrite oldest if full)
41	ring read all	Pure	All items chronologically
42	ring read last	Pure	Most recent N items
43	ring size	Pure	Current count
44	ring clear	Pure	Reset to empty
45	bitset create	Pure	Create named bitset
46	bitset set	Pure	Set bit at index
47	bitset clear bit	Pure	Clear bit at index
48	bitset test	Pure	Test bit at index
49	bitset count	Pure	Count set bits
50	bitset all set	Pure	All bits set test
51	bitset any set	Pure	Any bit set test
52	bitset reset	Pure	Clear all bits
53	bitset to list	Pure	Indices of set bits

Category 16: Path and Mount Operations

#	Primitive	Type	Description
54	path resolve	Pure	Dotted path to integer ID
55	path from id	Pure	Integer ID to dotted path
56	path parent	Pure	Parent path from path
57	path children	Pure	Child paths from path
58	path ancestors	Pure	All ancestor paths to root
59	path depth	Pure	Depth in tree
60	path exists	Pure	Test if path is registered
61	path common ancestor	Pure	Nearest common ancestor of two paths
62	kb mount	Pure (KB-internal)	Mount KB at path
63	kb unmount	Pure (KB-internal)	Remove mount point
64	kb mount info	Pure	Query mount details
65	kb list mounts	Pure	All mounts in KB
66	connection add	Pure (KB-internal)	Add connection to KB
67	connection remove	Pure (KB-internal)	Remove connection from KB
68	connection list	Pure	All connections from KB

#	Primitive	Type	Description
69	connection query	Pure	Find KBs by relationship type
70	connection graph	Pure	Build graph from all connections in scope

Category 17: Session Management Operations

#	Primitive	Type	Description
71	session snapshot	Pure (state capture)	Capture all live state
72	session restore	Pure (state restore)	Restore from snapshot
73	session reset	Pure (state clear)	Clear all live state
74	session clone	Pure (state copy)	Fork independent session
75	session kill	Pure (state destroy)	Destroy clone's live state
76	session list	Pure	List saved snapshots
77	session diff	Pure	Compare two snapshots
78	session info	Pure	Snapshot metadata

8.2 Revised Primitive Counts

Source	Pure	Operational	Total
VDR-6 (post regex removal)	211	44	255
VDR-8 data primitives	53	0	53
VDR-8 path and mount	17	0	17
VDR-8 session management	8	0	8
Combined total	289	44	333

All VDR-8 additions are pure primitives. They operate on KB-internal state deterministically. They do not interact with the external world. They do not require grants.

9. Constraint Integration

9.1 Data Primitive Constraints

Data primitives are subject to the same constraint system as everything else in the KB:

```

// Capacity constraint
constraint("queue_bounded", axiom, active,
condition(forall(queue(Q), queue_size(Q) =< queue_capacity(Q))),
on_violation("error"))

// Operational constraint on counter
constraint("max_retries", operational, active,
condition(counter_get("retry_count") < 5),
on_violation("escalate"))

// Lock timeout constraint
constraint("lock_timeout", operational, active,
condition(forall(lock(L), lock_held(L) implies lock_age(L) < 100)),
on_violation("force_release"))

// LRU minimum hit rate
constraint("cache_effective", operational, active,
condition(lru_hit_rate("recent_errors") > fraction(1, 5)),
on_violation("warn"))

```

9.2 Session Constraints

Session-level constraints govern snapshot and clone behavior:

```

// Maximum snapshots
constraint("snapshot_limit", operational, active,
condition(snapshot_count < 50),
on_violation("prune_oldest"))

// Clone drift detection

```

```

constraint("clone_max_age", operational, active,
condition(session_turns < 200),
on_violation("kill_and_reclone"))

// Clone error budget
constraint("clone_error_budget", operational, active,
condition(counter_get("errors") < 10),
on_violation("kill_and_reclone"))

```

9.3 Path Constraints

Path structure constraints prevent tree corruption:

```

// No orphan KBs
constraint("no_orphans", axiom, active,
condition(forall(kb(K), K = root or parent_exists(K))),
on_violation("error"))

// No mount cycles
constraint("no_mount_cycles", axiom, active,
condition(forall(mount(M), not(cycle_through(M)))),
on_violation("error"))

// Depth limit
constraint("depth_bounded", operational, active,
condition(forall(kb(K), path_depth(K) =< 16)),
on_violation("error"))

```

10. Integration Examples

10.1 LRU-Tracked Error Recovery

```
// LLM runs a test, it fails

CMD: ENV_EXEC(env_vdr, "python3", root.project.vdr.gyms.gym_25)

// ... task completes with failure ...

// LLM tracks the error

CMD: lru_push(root.project.vdr.recent_errors, "gym_25_fail", "maxflow BFS ti

CMD: counter_inc(root.project.vdr.error_count)

// Later, LLM checks recent errors before writing related code

CMD: lru_peek(root.project.vdr.recent_errors, 3)

// → [{gym_25_fail: "maxflow BFS timeout"}, {gym_24_fail: "graph cycle"}, ...

TEXT: "I see we've had graph-related failures recently.

    Let me check the BFS implementation before proceeding."
```

10.2 Queue-Managed Work

```
// User gives multiple requests

CMD: queue_push(root.project.vdr.pending_work, "implement cross-
entropy loss")

CMD: queue_push(root.project.vdr.pending_work, "fix maxflow BFS")

CMD: queue_push(root.project.vdr.pending_work, "add gym 26 for topology")

// LLM works through the queue

CMD: queue_pop(root.project.vdr.pending_work) → "implement cross-
entropy loss"

// ... does the work, commits results ...

CMD: queue_pop(root.project.vdr.pending_work) → "fix maxflow BFS"
```

```
// ... does the work ...

CMD: queue_size(root.project.vdr.pending_work) → 1

TEXT: "Two tasks done. One remaining: add gym 26 for topology."
```

10.3 Lock-Coordinated Long Operation

```
// Before starting training

CMD: lock_check(root.project.vdr.training_active) → false

CMD: lock_acquire(root.project.vdr.training_active, holder: "pretrain_v1")

CMD: ENV_EXEC(env_gpu, "python3 train.py", async=true) → task_050

// On subsequent turns, LLM checks lock before suggesting more training

CMD: lock_check(root.project.vdr.training_active) → true

TEXT: "Training is still running (started by pretrain_v1).

    Let's work on something else while we wait."

// When training task completes

CMD: lock_release(root.project.vdr.training_active)

CMD: counter_inc(root.project.vdr.completed_training_runs)
```

10.4 Session Snapshot and Clone

```
// User and LLM have built a well-configured session

CMD: session_snapshot("production_ready_v1")

TEXT: "Session state captured. All 12 active KBs,

    3 counters, 2 LRU caches, and current scope saved."

// Later, launching a disposable worker

CMD: session_clone("production_ready_v1", "worker_batch_001")
```

```
// Worker processes requests...

// Worker commits results to persistent KBs...

// Drift constraint fires after 150 turns

CMD: session_kill("worker_batch_001")

CMD: session_clone("production_ready_v1", "worker_batch_002")

TEXT: "Worker recycled. Fresh clone launched from stable operator."
```

10.5 Cross-Branch Reference With Mount

```
// VDR project needs C networking knowledge

CMD: kb_mount(root.project.vdr.refs.c_net,

               source: root.programming.c.networking,

               mode: read_only)

// Now queryable through the project's own path

CMD: kb_query(root.project.vdr.refs.c_net, socket_types)

// → returns facts from root.programming.c.networking

// The mount is a connection

CMD: connection_list(root.project.vdr)

// → [..., {target: root.programming.c.networking,

//         relationship: "mounted", direction: outbound}, ...]
```

10.6 Bitset Tracking Gym Completion

```
CMD: bitset_create(root.project.vdr.gyms.completed, width: 30)

// As gyms are completed:

CMD: bitset_set(root.project.vdr.gyms.completed, 1)
```

```

CMD: bitset_set (root.project.vdr.gyms.completed, 2)

// ...

CMD: bitset_set (root.project.vdr.gyms.completed, 25)

// Check progress:

CMD: bitset_count (root.project.vdr.gyms.completed) → 23

CMD: bitset_to_list (root.project.vdr.gyms.completed) → [1, 2, 3, 5, 6, 7, 8, 9, 10,

// Missing: 4, 23 — those gyms still need work

TEXT: "23 of 25 gyms complete. Gyms 4 and 23 still pending."

```

11. Falsification Criteria

F1. If any data primitive produces a different result from the same sequence of operations, the determinism guarantee is violated. This is testable by replaying operation sequences and comparing state.

F2. If a session restore produces a state that differs from the captured snapshot in any live state field — any counter value, any queue order, any LRU entry, any lock state — the snapshot mechanism is lossy. Testable by snapshot, modify, restore, compare.

F3. If a session clone's persistent KB assertions are not visible to other sessions sharing those KBs, the persistence boundary is wrong. Testable by asserting a fact in clone A and querying it in clone B.

F4. If a session clone's live state modifications (counter increments, queue pushes, lock acquisitions) are visible to other clones or the source snapshot, the isolation boundary is wrong. Testable by modifying live state in clone A and verifying clone B and the snapshot are unchanged.

F5. If a dotted path resolves to a different integer ID after system restart without explicit re-registration, the path registry has a consistency bug. Testable by recording path-to-ID mappings, restarting, and verifying.

F6. If a read-only mount allows write operations (fact assertion, data primitive mutation) to propagate to the source KB, the mount mode enforcement is broken. Testable by attempting writes through a read-only mount.

F7. If any data primitive exceeds its declared capacity without evicting (LRU, ring buffer) or rejecting (queue, stack), the growth bound is violated. Testable by filling to capacity and adding one more.

F8. If a KB export omits connections or data primitive state, the self-describing property is broken. Testable by exporting, importing to a fresh system, and comparing all fields.

F9. If a mount cycle is created without detection and rejection, the mount system has a vulnerability. Testable by attempting $A \rightarrow B \rightarrow A$ mount chains.

F10. If a command token referencing a dotted path that has been resolved to an integer ID produces a different result than the same command token with the path resolved freshly, the caching mechanism has a consistency bug. Testable by comparing cached and fresh resolution.

12. Implementation Priority

Phase	Component	Dependencies	Effort
1	Path registry and dotted naming	VDR-5 KB struct	Low
2	Integer ID assignment and resolution	Phase 1	Low
3	Slot ID registry for data primitives	Phase 2	Low
4	Counter and lock primitives	VDR-6 command tokens, Phase 2	Low
5	Queue and stack primitives	Phase 4	Low
6	LRU cache primitives	Phase 4	Medium
7	Ring buffer and bitset primitives	Phase 4	Low
8	KB struct update with all new fields	Phases 1-7	Medium
9	Connection field and operations	Phase 1	Low
10	Mount system with mode enforcement	Phase 2	Medium
11	Command token path resolution and caching	Phase 2	Medium
12	Session snapshot capture	Phase 8	Medium
13	Session restore and reset	Phase 12	Medium
14	Session clone and kill	Phase 13	Medium
15	Disposable clone lifecycle with drift constraints	Phase 14	Medium

Phase	Component	Dependencies	Effort
16	Mutation logging for data primitives	Phase 8	Low
17	Integration testing	All above	High

Phases 1-3 (path registry and ID systems) can begin immediately with the existing VDR codebase. Phases 4-7 (data primitives) are independent of the path system and can proceed in parallel. Phase 8 (KB struct update) integrates both tracks. Phases 12-15 (session management) require the data primitives and path system to be complete.

13. Conclusion

VDR-5 gave the system structured knowledge. VDR-6 gave it computational tools and execution environments. VDR-7 gave it a complete lifecycle. VDR-8 gives it runtime working memory, a universal addressing system, and session-level state management.

The data primitives — LRU caches, counters, locks, queues, stacks, ring buffers, bitsets — are fields on the KB struct, not a separate system. They scope with the KB. They inherit through the tree. They respect constraints. They log mutations. They are bounded by design. They are the LLM's working memory, giving it the same operational state tracking tools that any program would have, adapted to the turn-based execution model where nothing can block.

The dotted-path addressing system gives every KB, every data primitive, and every fact a structured name in a hierarchical namespace. Integer ID acceleration provides $O(1)$ runtime access. The LLM emits readable paths in command tokens. The executor resolves them to integers once. Everything downstream is fast integer array access.

Command tokens are reference selection, not text generation. The LLM picks a primitive name from a known vocabulary in the root KB and points at data by dotted path. Arguments are addresses, not values. The generative burden is minimal — selecting from a fixed set and assembling known references — which is exactly what language models are reliable at.

Session snapshots capture the complete live state atomically. Session clones fork independent working copies. Disposable operational clones maintain system stability through controlled recycling — build to a verified state, snapshot it, run workers off it, kill them before they drift, launch fresh ones from the same frozen baseline. The work persists in persistent KBs. The drift dies with the clone.

333 primitives across 20 categories. Dotted-path namespace with integer acceleration. Bounded data primitives in the KB struct. Session snapshots, clones, and disposable workers. Command tokens as compact reference assemblies. Everything in the KB. Everything queryable. Everything constrained. Everything addressable.

The runtime layer is complete.

Appendix A: Complete Data Primitive Specification

A.1 LRU Cache Specification

Property	Type	Constraint
Capacity	i32	$1 \leq \text{capacity} \leq 1000$
Key type	Text	Unique within cache
Value type	Term	Any VDR-Prolog term
Eviction	Automatic on push when full	Least recently accessed
Access ordering	Updated on get and push	Most recent first in peek
Deterministic	Same operations $\rightarrow$ same state	Required for snapshot consistency

A.2 Counter Specification

Property	Type	Constraint
Value type	i32	Within [min value, max value]
min value	i32	Default 0
max value	i32	Default $2^{31} - 1$
Overflow behavior	Clamp at bounds	No wraparound
Initial value	min value	Set on create

A.3 Lock Specification

Property	Type	Constraint
State	bool	held or free
Holder	?Text	Identifier of acquirer
Blocking	Never	Non-blocking by design
Acquire on held	Returns false	Does not wait
Timeout	Via constraint	lock age checked by constraint system

A.4 Queue Specification

Property	Type	Constraint
Capacity	i32	$1 \leq \text{capacity} \leq 1000$
Item type	Term	Any VDR-Prolog term

Property	Type	Constraint
Order	FIFO	Push to back, pop from front
Push when full	Returns false	Does not overwrite
Pop when empty	Returns none	Does not block

A.5 Stack Specification

Property	Type	Constraint
Capacity	i32	$1 \leq \text{capacity} \leq 1000$
Item type	Term	Any VDR-Prolog term
Order	LIFO	Push and pop at top
Push when full	Returns false	Does not overwrite
Pop when empty	Returns none	Does not block

A.6 Ring Buffer Specification

Property	Type	Constraint
Capacity	i32	$1 \leq \text{capacity} \leq 1000$
Item type	Term	Any VDR-Prolog term
Write when full	Overwrites oldest	Circular semantics
Read order	Chronological	Oldest first in read all

A.7 Bitset Specification

Property	Type	Constraint
Width	i32	$1 \leq \text{width} \leq 10000$
Bit type	bool	Set or clear
Index range	[0, width)	Out of range is error
Initial state	All clear	All bits false on create

Appendix B: Dotted-Path Grammar

B.1 Formal Grammar

```

path      := "root" ( "." segment ) *
segment   := alpha ( alpha | digit | "_" ) *

```


alpha := [a-z]

digit := [0-9]

B.2 Path Validation Rules

Rule	Condition	Example Valid	Example Invalid
Starts with root	path[0] = "root"	root.project	project.vdr
No empty segments	No ".." in path	root.a.b	root..b
No trailing dot	path[-1] ≠ "."	root.a	root.a.
Lowercase only	All alpha chars lowercase	root.project	root.Project
No leading digits	Segments start with alpha	root.vdr 1	root.1 vdr
Max depth	Segment count ≤ 16	root.a.b.c.d	(17 levels)
Max segment length	Each segment ≤ 64 chars	root.my project	(65-char segment)
Unique	No duplicate paths in registry	—	Two KBs with same path

B.3 Path Operations Reference

Operation	Input	Output	Example
resolve	path → i32	Integer ID	resolve("root.project.vdr") → 7
from id	i32 → path	Dotted path	from id(7) → "root.project.vdr"
parent	path → path	Parent path	parent("root.a.b") → "root.a"
children	path → []path	Child paths	children("root.a") → ["root.a.b", "root.a.c"]
ancestors	path → []path	Path to root	ancestors("root.a.b") → ["root.a", "root"]
depth	path → i32	Level count	depth("root.a.b") → 2

Operation	Input	Output	Example
exists	path $\rightarrow$ bool	Registration test	exists("root.a.b") $\rightarrow$ true
common ancestor	(path, path) $\rightarrow$ path	Shared prefix	common("root.a.b", "root.a.c") $\rightarrow$ "root.a"

Appendix C: Mount System Reference

C.1 Mount Record Structure

Field	Type	Required	Description
mount path	Text	Yes	Where the mount appears in the tree
mount id	i32	Yes	Integer ID of mount point
source path	Text	Yes	Dotted path of source KB
source id	i32	Yes	Integer ID of source KB
mode	enum	Yes	read only, read write, snapshot, mirror
created at	i32	Yes	Turn when mounted
created by	Text	Yes	Who created the mount
grant	?Text	Conditional	Required for read write mode

C.2 Mount Mode Behavior Matrix

Operation	read only	read write	snapshot	mirror
Query facts	Source KB live	Source KB live	Frozen copy	Source KB live
Assert fact	Blocked	To source KB	Blocked	Blocked
Retract fact	Blocked	From source KB	Blocked	Blocked
Read data primitives	Source KB live	Source KB live	Frozen copy	Source KB live
Mutate data primitives	Blocked	To source KB	Blocked	Blocked
Source changes visible	Yes	Yes	No (frozen)	Yes (automatic)
Requires grant	No	Yes (read write grant)	No	No

C.3 Mount Cycle Detection Algorithm

```

Rule: mount_safe(MountPath, SourcePath) :-
    not(path_prefix(SourcePath, MountPath)), // source is not under mount
    not(mount_chain_reaches(SourcePath, MountPath)). // no transitive cycle

Rule: mount_chain_reaches(From, Target) :-
    mount(From, _, Source, _),

    (Source = Target ; mount_chain_reaches(Source, Target)).

```

If mount_safe fails, the mount is rejected with a logged reason.

Appendix D: Session Management Reference

D.1 Session Snapshot Contents

Component	Captured	Size Estimate	Restoration
active scope	Array of i32 IDs	Tiny (~64 bytes)	Exact scope chain restored
active topic	Single i32	Tiny (4 bytes)	Exact topic restored
secondary scopes	Array of i32 IDs	Tiny (~64 bytes)	Exact secondary scopes restored
scratchpad	Ring buffer contents	Small (~4KB)	Exact scratchpad restored
counters (per KB)	Name→value map	Tiny per KB (~100 bytes)	Exact values restored
locks (per KB)	Name→state map	Tiny per KB (~100 bytes)	Exact states restored
lrus (per KB)	Full cache contents	Small per KB (~4KB)	Exact entries and order restored
queues (per KB)	Full queue contents	Small per KB (~2KB)	Exact items and order restored
stacks (per KB)	Full stack contents	Small per KB (~2KB)	Exact items and order restored
buffers (per KB)	Full ring buffer contents	Small per KB (~2KB)	Exact contents and write position
bitsets (per KB)	Bit array	Tiny per KB (~1KB)	Exact bits restored
working data (per KB)	All bindings	Variable	Exact bindings restored

Total snapshot size for a typical session: 10KB–100KB depending on active KB count and data primitive contents. Snapshots are small because they capture state, not knowledge.

D.2 Session Operation Semantics

Operation	Effect on Persistent KBs	Effect on Live State	Effect on Other Sessions
snapshot	None	Read (captured)	None
restore	None	Overwritten entirely	None
reset	None	Cleared to defaults	None
clone	None	Copied to new session	None
kill	None	Destroyed for target	None

No session operation modifies persistent KBs. Session operations affect only the live state of the target session.

D.3 Clone Isolation Properties

Data Type	Shared Between Clones	Isolated Per Clone
KB facts	Yes (persistent, shared)	—
KB rules	Yes (persistent, shared)	—
KB constraints	Yes (persistent, shared)	—
KB connections	Yes (persistent, shared)	—
Counters	—	Yes (independent copies)
Locks	—	Yes (independent copies)
LRU caches	—	Yes (independent copies)
Queues	—	Yes (independent copies)
Stacks	—	Yes (independent copies)
Ring buffers	—	Yes (independent copies)
Bitsets	—	Yes (independent copies)
Working data	—	Yes (independent copies)
Scratchpad	—	Yes (independent copies)

D.4 Disposable Clone Lifecycle

Step	Operation	Trigger	Side Effect
1	session clone(source, name)	Manual or automated	New session with independent live state
2	Clone processes requests	User or API traffic	Persistent KB assertions visible to all

Step	Operation	Trigger	Side Effect
3	Drift constraint checked	Every turn	Constraint evaluation
4a	No drift detected	Constraint satisfied	Continue processing
4b	Drift detected	Constraint violated	Proceed to step 5
5	session kill(name)	Constraint violation handler	Live state destroyed
6	session clone(source, new name)	Automated reclone	Fresh clone from same snapshot
7	Return to step 2	—	Cycle continues

Appendix E: Command Token Resolution Reference

E.1 Resolution Pipeline

Stage	Input	Output	Cost
1. Parse	Raw command token text	Primitive name + arg list	O(token length)
2. Primitive lookup	Primitive name string	Primitive ID (i32)	O(1) hash lookup
3. Path resolution	Dotted path strings	KB IDs (i32)	O(1) hash lookup per path
4. Slot resolution	Primitive name within KB	Slot ID (i32)	O(1) hash lookup
5. Dispatch	Primitive ID + resolved args	Execution	O(1) dispatch
6. Result storage	Result + store result path	KB update	O(1) via resolved ID

Total resolution overhead: 3-4 hash lookups per command token, followed by direct integer-indexed access for the actual operation.

E.2 Path Resolution Cache

Property	Specification
Cache scope	Per turn
Cache key	Dotted path string
Cache value	Integer ID

Property	Specification
Eviction	Cleared at turn boundary
Size	Bounded by unique paths referenced per turn
Invalidation	If KB tree changes mid-turn (rare), cache is flushed

E.3 Command Token Size Comparison

Approach	Example	Token Count	Failure Modes
Freeform JSON generation	<code>{"function": "list", "arguments": {"list": [47, 3, 91, 15], "order": "ascending"}}</code>	~30 tokens	Malformed JSON, wrong keys, type errors
Current LLM function calling	<code>list(sort(list=[47, 3, 91, 15], order="ascending"))</code>	~15 tokens	Wrong argument names, misspelled function
VDR-8 reference-based	<code>CMD: list(sort(root.project.data.unsorted.ascending))</code>	~8 tokens	Wrong path (but validated)
VDR-8 with integers (internal)	<code>CMD: 23(7.4, 1)</code>	~5 tokens	N/A (not LLM-generated, post-resolution)

The LLM generates the reference-based form (~8 tokens). The executor resolves to the integer form (~5 values). The generative burden is $2-4 \times$ lower than standard function calling and the failure surface is smaller because the components are validated references, not generated syntax.

Appendix F: Revised Cumulative Statistics

F.1 Complete Paper Series

Paper	Registry	Central Result
VDR-1	[HOWL-VDR-1-2026]	Exact arithmetic in irreducible triple form
VDR-2	[HOWL-VDR-2-2026]	15 domains, 282 tests
VDR-3	[HOWL-VDR-3-2026]	23 domains, transcendental integration
VDR-4	[HOWL-VDR-4-2026]	24-module ML stack, working exact transformer
VDR-5	[HOWL-VDR-5-2026]	Prolog KB architecture, constraints, scoped knowledge
VDR-6	[HOWL-VDR-6-2026]	255 primitives, command tokens, operational environments
VDR-7	[HOWL-VDR-7-2026]	12-phase lifecycle, training through retirement
VDR-8	[HOWL-VDR-8-2026]	Data primitives, dotted paths, session management

F.2 Complete Module Count

Layer	Count	Source
Arithmetic	5	VDR-1
Transcendental	3	VDR-4
ML	4	VDR-4
Infrastructure	8	VDR-4
Architecture	4	VDR-4
Logic/Knowledge	3	VDR-5
Execution	3	VDR-6
Lifecycle	4	VDR-7
Runtime state	3 (new)	VDR-8
Total	37	

New VDR-8 modules: `data_primitives.py` (LRU, counter, lock, queue, stack, ring buffer, bitset implementations), `pathing.py` (dotted-path registry, integer ID assignment, mount system, connection operations), `sessions.py` (snapshot capture, restore, reset, clone, kill, diff).

F.3 Complete Primitive Count

Source	Pure	Operational	Total
VDR-6 (post regex removal)	211	44	255
VDR-8 data primitives	53	0	53
VDR-8 path and mount	17	0	17
VDR-8 session management	8	0	8

Source	Pure	Operational	Total
Total	289	44	333

F.4 Complete KB Struct Fields

Field	Source	Type	Classification
name	VDR-5	Text	Identity
path	VDR-8	Text	Identity
id	VDR-8	i32	Identity
facts	VDR-5	FactSet	Persistent
rules	VDR-5	RuleSet	Persistent
constraints	VDR-5 addendum	[]Constraint	Persistent
connections	VDR-8	[]Connection	Persistent
working data	VDR-5	?WorkingDataSet	Live
lrus	VDR-8	HashMap(Text, LRU)	Live
counters	VDR-8	HashMap(Text, Counter)	Live
locks	VDR-8	HashMap(Text, LockState)	Live
queues	VDR-8	HashMap(Text, BoundedQueue)	Live
stacks	VDR-8	HashMap(Text, BoundedStack)	Live
buffers	VDR-8	HashMap(Text, RingBuffer)	Live
bitsets	VDR-8	HashMap(Text, Bitset)	Live
parent id	VDR-5/8	?i32	Structural
children ids	VDR-5/8	[]i32	Structural
mounts	VDR-8	[]Mount	Structural
visibility	VDR-5	enum	Metadata
frozen	VDR-5	bool	Metadata
owner	VDR-5 addendum	?Text	Metadata
created at	VDR-5	i32	Metadata
last modified	VDR-5	i32	Metadata

24 fields. 5 persistent. 8 live. 5 structural. 6 metadata.

F.5 Complete Test Plan

Source	Existing Tests	Planned Tests	Total
VDR-1 through VDR-4	705	—	705
VDR-6 pure primitives	—	615	615
VDR-6 operational	—	132	132
VDR-7 lifecycle	—	200	200
VDR-8 data primitives (53×3)	—	159	159
VDR-8 path operations (17×3)	—	51	51
VDR-8 session operations (8×3)	—	24	24

Source	Existing Tests	Planned Tests	Total
VDR-8 integration	—	40	40
Total	705	1221	1926

END HOWL-VDR-8-2026

Registry: [[HOWL-VDR-8-2026](#)]

Status: Specification complete

Domain: Applied Philosophy / Systems Architecture / Runtime State Management

Central Result: Runtime data primitives (LRU, counter, lock, queue, stack, ring buffer, bitset) as KB struct fields. Universal dotted-path addressing with integer ID acceleration. Command tokens as compact reference assemblies. Session snapshots, cloning, and disposable operational clones. 333 total primitives across 20 categories.

Foundation: VDR-1 through VDR-7, MATH-3, MATH-4

Key Principles: (1) Data primitives belong in the KB struct, not in a separate system. (2) The dotted path is the human API; the integer ID is the machine API. (3) Command tokens are reference selection, not text generation. (4) Live state is capturable, restorable, cloneable, and disposable. (5) Everything is a KB.

Falsification: Ten specific criteria testable by exact comparison, operation replay, and isolation verification.

VDR-8 Extended Appendix Tables

Complete Reference Material for Runtime State, Data Pathing, and Session Management

Appendix G: Data Primitive Interaction Patterns

G.1 Multi-Primitive Coordination Patterns

Pattern	Primitives Used	Workflow	Use Case
Retry with backoff	counter + lock + lru	Counter tracks attempts, lock prevents concurrent retries, LRU stores failure reasons	Flaky test re-execution
Work queue with progress	queue + counter + bitset	Queue holds pending items, counter tracks completed, bitset marks which indices done	Batch processing gyms
Rolling error window	ring buffer + counter	Ring stores recent N errors, counter tracks total errors across session	Drift detection for clones
Undo/redo	stack + stack	One stack for undo history, one for redo future, push/pop on user action	Interactive KB editing
Priority staging	queue + queue + lru	Urgent queue, normal queue, LRU of recently completed to avoid repeats	Multi-priority request handling
Mutex-like exclusion	lock + queue	Lock guards the resource, queue buffers waiting requests for next turn	Sequential access to shared environment
Checkpoint tracking	bitset + counter + stack	Bitset marks completed phases, counter tracks current phase, stack holds rollback points	Multi-phase pipeline progress

Pattern	Primitives Used	Workflow	Use Case
Cache-assisted lookup	lru + counter	LRU caches recent query results, counter tracks hit/miss rate	Repeated KB queries across turns

G.2 Data Primitive Cross-Reference by KB Type

KB Type (from VDR-7)	Useful Primitives	Typical Configuration	Purpose
Training run	counter(step, epoch), lock(training active), ring(recent losses), lru(anomaly cache)	step: max=1M, ring: cap=100, lru: cap=50	Track training progress and anomalies
Evaluation suite	bitset(benchmarks completed), counter(total passed), queue(pending benchmarks)	bitset: width=num benchmarks, queue: cap=50	Track evaluation progress
Deployment	lock(update in progress), counter(requests served), ring(recent errors), lru(recent requests)	ring: cap=200, lru: cap=100	Operational state during serving
Feedback collection	counter(judgments received), queue(pending review), bitset(annotators active)	queue: cap=100, bitset: width=max annotators	Track annotation progress
Source registry	counter(sources registered), lru(recent acquisitions), lock(acquisition active)	lru: cap=20	Data sourcing state

KB Type (from VDR-7)	Useful Primitives	Typical Configuration	Purpose
Corpus preparation	bitset(pipeline steps complete), counter(documents processed), ring(recent rejections)	bitset: width=7 (one per pipeline step)	Pipeline progress tracking
Monitoring	ring(latency samples), ring(error samples), counter(safety flags), lru(flagged interactions)	ring: cap=500, lru: cap=100	Runtime metric buffering
Project	queue(pending tasks), stack(undo history), lru(recent results), counter(completed tasks)	queue: cap=50, stack: cap=30, lru: cap=50	Developer workflow state
Session management	counter(clone count), counter(snapshot count), lru(recent clones), bitset(active clones)	bitset: width=max concurrent clones	Clone lifecycle tracking

G.3 Data Primitive Inheritance Behavior

Scenario	Parent KB Has	Child KB Has	Query From Child Returns	Rationale
No shadow	counter("errors",—5)		5	Inherited from parent
Child shadows	counter("errors",5)	counter("errors",0)		Child overrides parent
Different names	counter("global errors", 5)	counter("local errors", 2)	Both visible by name	No conflict, both accessible
Parent locked	lock("training", —held)		held	Lock state inherited

Scenario	Parent KB Has	Child KB Has	Query From Child Returns	Rationale
Child overrides lock	lock(“training”, held)	lock(“training”, free)	free	Child’s local state wins
Parent LRU	lru(“cache”, [...])	—	Parent’s entries	Read-through to parent
Child LRU shadows	lru(“cache”, [...parent...])	lru(“cache”, [...child...])	Child’s entries	Child cache is independent

Inheritance rule: data primitives inherit by name through the KB tree, same as working data bindings (VDR-5 Section 6.2). Child values shadow parent values. Lookup walks from current KB to root. First match wins.

Appendix H: Integer ID Allocation and Lifecycle

H.1 ID Assignment Rules

Rule	Description	Rationale
Sequential	IDs assigned in creation order starting from 0	Simple, predictable
Root is 0	root path always gets ID 0	Convention, fast root checks
Never reused	Deleted KB’s ID is retired permanently	Prevents stale references
Stable across sessions	Same path always maps to same ID within a system instance	Snapshot consistency
Gapless at startup	Initial batch assignment is contiguous	Array-friendly
Gaps after deletion	Retired IDs create gaps in the ID space	Acceptable — array is sparse
Registry is persistent	path to id mapping survives restart	Required for F5 falsification
Max ID	$2^{31} - 1$ (max i32)	Sufficient for any practical tree

H.2 Slot ID Assignment Rules

Rule	Description	Rationale
Per-KB namespace	Slot IDs are unique within one KB, not globally	Compact local addressing
Sequential per KB	Slots assigned 0, 1, 2, ... within each KB	Simple
Cross-type	All data primitive types share one slot namespace per KB	counter("a")=0, lru("b")=1, lock("c")=2
Never reused within KB	Deleted primitive's slot is retired	Prevents stale references
Type tagged	Slot registry records which type each slot is	Prevents counter operations on a queue slot

H.3 ID Space Capacity Estimates

Tree Scale	KBs	IDs Used	Slots Per KB (avg)	Total Addressable
Small project	50	50	5	250
Medium project	500	500	10	5,000
Full lifecycle (VDR-7)	2,000	2,000	15	30,000
Large organization	10,000	10,000	20	200,000
Theoretical maximum	2 ³¹	2 ³¹	2 ³¹ per	2 ⁶² (unreachable)

All addressing fits in two i32 values. Memory overhead for the path registry: approximately 100 bytes per KB (path string + ID mapping). For 10,000 KBs: ~1MB.

H.4 ID Registry Serialization

```
RegistryRecord = struct {
    id: i32,
    path: Text,
    status: enum { active, retired },
    created_at: i32,
    retired_at: ?i32,
};
```

The registry serializes as a flat array of records, indexed by ID. Retired entries remain in the array with status=retired. The registry is a persistent KB fact:

`root.system.path_registry`

Appendix I: Dotted-Path Relative Addressing

I.1 Relative Path Operators

While all stored paths are absolute (starting from `root`), the command token parser supports relative path shorthand that resolves against the active scope:

Operator	Meaning	Example (active: <code>root.project.vdr</code>)	Resolves To
<code>.name</code>	Child of active KB	<code>.training</code>	<code>root.project.vdr.training</code>
<code>..</code>	Parent of active KB	<code>..</code>	<code>root.project</code>
<code>..name</code>	Sibling of active KB	<code>..math</code>	<code>root.project.math</code>
<code>...</code>	Grandparent	<code>...</code>	<code>root</code>
<code>@root</code>	Explicit root	<code>@root.sessions</code>	<code>root.sessions</code>
<code>@self</code>	Active KB itself	<code>@self</code>	<code>root.project.vdr</code>
<code>@mount.name</code>	Through mount point	<code>@mount.c net</code>	(resolved mount target)

I.2 Resolution Rules

Priority	Rule	Description
1	Absolute path	If starts with “ <code>root.</code> ”, treat as absolute
2	@-prefix	If starts with “ <code>@</code> ”, resolve special prefix
3	Relative path	If starts with “ <code>.</code> ” or “ <code>..</code> ”, resolve against active scope
4	Bare name	If no dots or prefixes, search active KB’s children first, then scope chain

Relative paths are resolved to absolute paths before integer ID lookup. The resolved absolute path is what gets cached and logged. Relative paths never appear in KB facts, connections, or snapshots — they are a command token convenience only.

I.3 Path Wildcards for Bulk Operations

Wildcard	Meaning	Example	Matches
*	All children	<code>root.project.vdr</code>	All direct children of <code>root.project.vdr</code>
**	All descendants	<code>root.project.vdr</code>	All KBs in the subtree
*.name	Named children of all siblings	<code>root.project.training.kbs</code>	Training KBs across all projects

Wildcards are valid in query and bulk operations (archive, export, constraint check) but not in assertion or mutation operations. You cannot assert a fact into a wildcard path.

Appendix J: Connection Relationship Taxonomy

J.1 Standard Relationship Types

Relationship	Direction Convention	Lifecycle Phase	Example
sourced from	inbound to corpus from source	1→2	corpus ← source wikipedia
tokenized into	outbound from corpus to tokenized	2→3	corpus → tokenized corpus
tokenized by	inbound to tokenized from vocab	3	tokenized corpus ← vocab
initialized from	inbound to init from arch	4	model init ← model arch
trained from	inbound to run from corpus + init	4→5	training run ← model init
checkpointed at	outbound from run to checkpoint	5	training run → checkpoint
finetuned from	inbound to finetune from checkpoint	5→6	finetune run ← checkpoint
feedback for	inbound to feedback from model	6→7	checkpoint feedback ← checkpoint

Relationship	Direction Convention	Lifecycle Phase	Example
reward trained on	inbound to reward from feedback	7	reward model ← feedback
aligned by	inbound to aligned checkpoint from RLHF/DPO	7	checkpoint safe ← rlhf run
evaluated by	outbound from model to eval	8	checkpoint → eval result
deployed as	outbound from model to deployment	8→9	checkpoint → deployment
monitored by	outbound from deployment to monitoring	9→10	deployment → monitoring
updated by	inbound to deployment from update	11	deployment ← update
superseded by	outbound from old deployment to new	11	deployment v1 → deployment v2
retired as	outbound from model to retirement	12	checkpoint → retirement
references	bidirectional	Any	zig.networking ↔ c.networking
mounted from	inbound to mount point from source	Any	mount point ← source kb
cloned from	inbound to clone from snapshot	Session	clone ← snapshot

J.2 Connection Constraint Patterns

Constraint	Condition	Type	Purpose
No orphan KBs	Every non-root KB has at least one inbound connection	operational	Tree integrity
Deployment requires evaluation	Every deployment KB has inbound connection of type evaluated by	operational	Quality gate

Constraint	Condition	Type	Purpose
Source requires license	Every source KB has a license fact	legal	Legal compliance
No dangling connections	Every connection target exists in path registry	axiom	Reference integrity
Bidirectional consistency	If $A \rightarrow B$ exists, $B \leftarrow A$ should exist	operational	Graph completeness
Lifecycle ordering	sourced from precedes trained from precedes deployed as	operational	Phase sequencing

J.3 Connection Graph Queries

Query	Graph Operation	Primitives Used	Returns
Full lineage to root	Backward BFS from target	connection graph + graph bfs	Ordered ancestor list
All dependents of a source	Forward BFS from source	connection graph + graph bfs	All downstream KBs
Impact of removing a KB	Forward reachability	graph connected components	Affected KB set
Shortest path between KBs	Path search on connection graph	graph shortest path	Path through KB tree
Cycle detection	Cycle search on directed graph	graph cycle detect	Boolean + cycle path if found
Disconnected subgraphs	Component analysis	graph connected components	List of isolated KB clusters
Critical path (lifecycle)	Longest path through lifecycle connections	graph topological sort	Phase ordering

Appendix K: Mount Use Case Reference

K.1 Common Mount Patterns

Pattern	Mount Mode	Source Branch	Target Branch	Purpose
Shared reference library	read only	root.reference.*	root.project.. <i>refs</i> .	Projects reference shared knowledge
Shared test fixtures	read only	root.testing.fixtures	root.project.*.test.fixtures	Projects use common test data
Collaborative editing	read write	root.team.shared kb	root.user.*.collab	Multiple users edit same KB
Stable dependency	snapshot	root.external.library v2	root.project.deps.library	Freeze dependency at known version
Live dashboard data	mirror	root.deployment.monitoring	root.admin.dashboard.admin	Admin sees live metrics
Cross-project learning	read only	root.project.alpha.results	root.project.beta.refs.alpha	Project B references Project A's results
Archive access	read only	root.archive.model v1	root.project.current.refs.model	Current project references archived model

K.2 Mount Permission Requirements

Mode	Grant Required	Constraint Checked	Logged As
read only	Source KB visibility $\geq$ user's access level	mount safe (no cycles)	mount created(path, source, read only, turn)
read write	Source KB visibility + explicit write grant	mount safe + write grant valid	mount created(path, source, read write, turn, grant)

Mode	Grant Required	Constraint Checked	Logged As
snapshot	Source KB visibility	mount safe	mount created(path, source, snapshot, turn) + snapshot fact
mirror	Source KB visibility	mount safe	mount created(path, source, mirror, turn)

K.3 Mount Impact on Scope Resolution

When a mount point is in the active scope chain, queries resolve through the mount:

Scope Chain	Query	Resolution Path	Result Source
[self, parent, ..., root]	Fact query	Self first, then parent, then mount if in chain	First match in scope order
Mount in scope	Fact in mounted KB	Mount resolves to source, query hits source	Source KB via mount
Mount is read only	Assert fact	Blocked at mount point	Error: read-only mount
Mount is snapshot	Source changed	Snapshot is stale (by design)	Frozen snapshot, not live source
Multiple mounts	Same predicate in two mounts	First mount in scope order wins	Scope priority determines winner

Appendix L: Session Snapshot Size Estimates

L.1 Per-Primitive State Sizes

Primitive	Per-Entry Size	Typical Capacity	Typical Full Size	Empty Size
LRU entry	~80 bytes (key + value + timestamp)	50	4KB	16 bytes
Counter	4 bytes (i32 value)	1 (single value)	4 bytes	4 bytes

Primitive	Per-Entry Size	Typical Capacity	Typical Full Size	Empty Size
Lock	~40 bytes (state + holder + timestamp)	1 (single state)	40 bytes	8 bytes
Queue item	~40 bytes (Term value)	50	2KB	16 bytes
Stack item	~40 bytes (Term value)	30	1.2KB	16 bytes
Ring buffer entry	~40 bytes (Term value)	100	4KB	16 bytes
Bitset	width/8 bytes	100 bits	13 bytes	13 bytes
Working data binding	~60 bytes (key + value + metadata)	50	3KB	16 bytes

L.2 Snapshot Size by Session Complexity

Session Type	Active KBs	Primitives Per KB (avg)	Total Primitives	Estimated Snapshot Size
Simple conversation	5	2	10	2KB
Single project work	15	5	75	20KB
Multi-project with lifecycle	30	8	240	60KB
Full deployment monitoring	50	12	600	150KB
Maximum practical	100	15	1500	400KB

Snapshots are small. Even a heavily loaded session with 100 active KBs and 1500 data primitives produces a snapshot under 500KB. For comparison, a single model checkpoint from VDR-7 is megabytes to gigabytes. Session snapshots capture working memory, not knowledge.

L.3 Snapshot Operation Timing Estimates

Operation	Dominant Cost	Estimated Duration	Scales With
session snapshot	Iterate active KBs, copy live state	<10ms for typical session	Active KB count × primitives per KB
session restore	Overwrite live state from snapshot	<10ms for typical session	Same as snapshot
session reset	Zero/clear all live state	<5ms	Active KB count × primitives per KB
session clone	Copy snapshot to new session namespace	<10ms + snapshot cost	Same as snapshot
session kill	Deallocate clone's live state	<5ms	Clone's KB count
session diff	Compare two snapshots field by field	<20ms	Sum of both snapshots' primitives

All operations are bounded by the number of active KBs and their data primitive counts. With bounded capacities on all primitives, the total state to process is always small.

Appendix M: Clone Drift Detection Reference

M.1 Drift Indicators

Indicator	Measurement	Healthy Range	Warning Threshold	Kill Threshold
Session age (turns)	counter get("session turns")	0-150	150	200
Context usage	context tokens / max context	0-80%	80%	90%
Working denominator growth	max denominator across working data	$< 2^{40}$	2^{40}	2^{48}
Error accumulation	counter get("errors") / counter get("requests")	$< 1\%$	2%	5%

Indicator	Measurement	Healthy Range	Warning Threshold	Kill Threshold
Scratchpad saturation	ring size("scratchpad") / capacity	0-70%	70%	90%
Constraint violation rate	counter get("violations") / counter get("checks")	0%	1%	5%
Response latency trend	ring read last("latency", 10) trend	Stable or decreasing	20% increase	50% increase
LRU eviction rate	counter get("evictions") / counter get("pushes")	< 50%	70%	90%

M.2 Drift Response Actions

Severity	Condition	Action	Automatic?
Nominal	All indicators healthy	Continue	—
Watch	Any indicator at warning	Log, increase monitoring frequency	Yes
Warn	Multiple indicators at warning	Alert operator, prepare for reclone	Yes (alert), No (action)
Reclone	Any indicator at kill threshold	Kill clone, launch fresh from snapshot	Yes
Escalate	Reclone fails or new clone immediately drifts	Alert operator, suspend automatic cloning	Yes (alert), No (resolution)
Investigate	Repeated escalation	Operator reviews stable operator snapshot	No (manual)

M.3 Clone Lifecycle Metrics

Metric	Tracked By	Stored In	Query Pattern
Clone lifespan (turns)	counter per clone	session management KB	Histogram of clone lifespans
Clones launched per snapshot	counter per snapshot	session management KB	Rate of recycling
Mean useful work per clone	assertions committed / turns lived	session management KB	Efficiency metric
Drift cause distribution	categorized kill reasons	session management KB	Most common drift modes
Time between reclones	timestamp deltas	session management KB	Stability trending
Snapshot version usage	which snapshot each clone used	session management KB	Snapshot lineage analysis

Appendix N: Command Token Error Handling Reference

N.1 Command Token Validation Stages

Stage	Check	Error Type	Recovery
1. Syntax	Well-formed CMD token structure	Parse error	Log, skip command, continue text output
2. Primitive exists	Primitive name found in root.primitives KB	Unknown primitive	Log, suggest similar primitive names
3. Path exists	All dotted paths resolve in registry	Unknown path	Log, report which path failed
4. Slot exists	Data primitive name exists in target KB	Unknown slot	Log, report available primitives in KB
5. Type match	Arguments match primitive signature types	Type mismatch	Log, report expected vs actual types
6. Capacity check	Operation won't exceed primitive bounds	Capacity exceeded	Reject push (queue/stack), evict (LRU/ring)

Stage	Check	Error Type	Recovery
7. Mount mode	Write operation permitted by mount mode	Permission denied	Log, report mount is read-only
8. Constraint check	No constraint blocks the operation	Constraint violation	Execute constraint's on violation action

N.2 Error Fact Structure

```
Fact: command_error(
  turn(47),
  command("counter_inc(root.project.vdr.nonexistent.retry_count)"),
  stage("path_resolution"),
  error("path_not_found"),
  detail("root.project.vdr.nonexistent does not exist in path registry"),
  recovery("command_skipped"),
  timestamp(...)
).
```

Error facts are asserted into the active session KB and are visible in the scratchpad. The LLM can query recent errors to adjust its behavior:

```
CMD: lru_peek(root.system.recent_command_errors, 3)
```

N.3 Error Rate Constraints

```
constraint("command_error_budget", operational, active,
  condition(counter_get("command_errors") < counter_get("commands_issued") * f
  on_violation("warn"))
constraint("consecutive_error_limit", operational, active,
```

```
condition(counter_get("consecutive_errors") < 3),  
on_violation("pause_and_replan"))
```

Appendix O: Data Primitive Mutation Log Reference

O.1 Mutation Event Types

Primitive Type	Operation	Event Type	Fields Logged
LRU	push	lru push	kb id, name, key, value, evicted key (if any)
LRU	get (hit)	lru hit	kb id, name, key
LRU	get (miss)	lru miss	kb id, name, key
LRU	evict	lru evict	kb id, name, key, reason
LRU	clear	lru clear	kb id, name, entry count cleared
Counter	inc/dec/add	counter change	kb id, name, old value, new value, delta
Counter	reset	counter reset	kb id, name, old value
Counter	set	counter set	kb id, name, old value, new value
Lock	acquire	lock acquire	kb id, name, holder, success
Lock	release	lock release	kb id, name, old holder
Lock	force release	lock force	kb id, name, old holder, forced by
Queue	push	queue push	kb id, name, item, new size, success
Queue	pop	queue pop	kb id, name, item, new size
Queue	clear	queue clear	kb id, name, items cleared
Stack	push	stack push	kb id, name, item, new size, success
Stack	pop	stack pop	kb id, name, item, new size
Stack	clear	stack clear	kb id, name, items cleared
Ring	write	ring write	kb id, name, item, overwrote (if full)

Primitive Type	Operation	Event Type	Fields Logged
Ring	clear	ring clear	kb id, name, items cleared
Bitset	set	bitset set	kb id, name, index, was already set
Bitset	clear	bitset clear bit	kb id, name, index, was set
Bitset	reset	bitset reset	kb id, name, bits cleared

O.2 Mutation Log Retention Policy

Policy	Retention	Rationale
Current turn	Always retained	Active debugging
Previous 5 turns	Retained	Recent context
Older than 5 turns	Pruned unless flagged	Memory management
Error-related mutations	Retained for 20 turns	Debugging errors needs history
Lock mutations	Retained for lock lifetime + 5 turns	Audit lock usage
Flagged by user	Retained indefinitely	Explicit preservation

O.3 Mutation Log Query Patterns

Query	Purpose	Example
All mutations on a specific primitive	Audit trail	mutations(kb id=7, name="retry count")
All mutations in a turn	Turn replay	mutations(turn=47)
All lock acquisitions by holder	Access audit	mutations(type=lock acquire, holder="training v1")
All evictions from an LRU	Cache pressure analysis	mutations(type=lru evict, name="recent errors")
Counter value at specific turn	Historical state reconstruction	counter value at(kb=7, name="errors", turn=30)
Queue contents at specific turn	Historical state reconstruction	Replay pushes and pops from mutation log

Appendix P: Scratchpad as Ring Buffer Specification

P.1 Scratchpad Respecification

The scratchpad, introduced informally in VDR-6, is formally specified in VDR-8 as a ring buffer on the session KB:

```
Location: root.sessions.active.scratchpad

Type: RingBuffer

Capacity: Configurable (default 100 entries)

Entry type: ScratchpadEntry

ScratchpadEntry = struct {
  turn: i32,

  entry_type: enum { pure_fn_call, kb_query, constraint_check,
                    grant_check, reasoning_step, plan_step,
                    error_recovery, command_result },

  content: Term,

  timestamp: i32,

};
```

P.2 Scratchpad Relationship to Other Primitives

Aspect	Scratchpad	Other Ring Buffers	Other LRUs
Location	root.sessions.active.scratchpad	Any KB	Any KB
Created by	System (automatic)	LLM (explicit command)	LLM (explicit command)
Written by	Command token executor	LLM (explicit command)	LLM (explicit command)
Read by	LLM (scratchpad queries), owner (/show)	LLM (ring read *), owner	LLM (lru get/peek), owner
Cleared on	session reset	session reset	session reset
Captured by	session snapshot	session snapshot	session snapshot

Aspect	Scratchpad	Other Ring Buffers	Other LRUs
Purpose	Computation trace	Application-specific rolling data	Application-specific cached data

The scratchpad is not special — it is a ring buffer that happens to be auto-created on session start and auto-written by the command executor. The LLM can create additional ring buffers for its own purposes.

Appendix Q: Working Data Reclassification

Q.1 Working Data as Live State

VDR-5 introduced working data sets as scoped key-value bindings on topic KBs. VDR-8 classifies working data as live state — captured by session snapshots, cleared by session reset (optionally), and isolated per clone.

This reclassification clarifies an ambiguity in VDR-5: are working data bindings persistent or ephemeral? The answer: they are live state by default. If a working data binding should survive session reset, it should be promoted to a KB fact through KB_ASSERT.

Q.2 Working Data Reset Modes

Mode	On session reset	On session restore	On session kill	Rationale
clear all (default)	All bindings cleared	Overwritten from snapshot	Destroyed	Clean slate
preserve flagged	Flagged bindings retained	Overwritten from snapshot	Destroyed	Keep explicitly marked values
preserve all	All bindings retained	Overwritten from snapshot	Destroyed	Soft reset (clear primitives, keep data)

The mode is configurable per session. The default (clear_all) provides the cleanest reset semantics.

Q.3 Promotion Path: Live → Persistent

Live State	Promotion Command	Persistent Form	When To Promote
Working data binding	KB ASSERT(kb, fact)	KB fact	Value is a permanent result, not working state
Counter final value	KB ASSERT(kb, count fact)	KB fact	Completed count is a result worth keeping
Queue contents (remaining)	KB ASSERT per item + clear	KB facts	Remaining work items should survive reset
LRU contents	KB ASSERT per entry + clear	KB facts	Cached items are worth persisting
Bitset completion state	KB ASSERT(kb, completion fact)	KB fact	Progress record is a result
Lock state	Not typically promoted	—	Locks are inherently transient

The clone workflow relies on this pattern: the clone works using live state, promotes results to persistent KBs via KB_ASSERT, and the live state is discarded on kill. The persistent assertions survive.

Appendix R: Scope Resolution with Mounts — Full Algorithm

R.1 Scope Resolution Steps

```
resolve_query(predicate, args) → result:
```

```
1. Determine active scope chain:
```

```
    chain = [active_kb_id] + ancestors(active_kb_id) + [0] // self to root
    chain += secondary_scope_ids // explicitly activated
```

```
2. For each kb_id in chain (in order):
```

```
    a. Check kb_id's own facts for match
```

```
    b. If kb_id has mounts:
```

```
        For each mount in kb_id.mounts:
```

```
            If mount is in scope (mount_path is descendant of chain member):
```

```

    Resolve mount to source_id

    Check source KB's facts for match

    (Respect mount mode: skip if query is write and mode is read_only)

c. If match found: return result with source KB tagged

d. If no match: continue to next kb_id in chain

3. If no match in entire chain: return not_found

4. For cross-scope query (query_across):

    Iterate ALL registered KBs (not just scope chain)

    Tag each result with its source kb_id

    Return full tagged result set

```

R.2 Scope Resolution Examples

Active KB	Query	Chain Walked	Mount Traversed	Result Source
root.project.vdr.training.run 01	at(step 100)	run 01 → training → vdr → project → root	None	run 01 (local fact)
root.project.vdr	socket types	vdr → project → root	root.project.vdr.refs.c.net → root.programming.c.networking	c.networking (via mount)
root.stories.london	dob age	london → stories → root	None	london.characters (child searched)
root.project.vdr	eval result	vdr → project → root	None	not found (eval is in different branch)
root.project.vdr	eval result (query across)	ALL KBs	All mounts	Tagged results from all eval KBs

Appendix S: Data Primitive Test Plan

S.1 LRU Tests (24 tests)

Test	Input	Expected	Tests Property
Create empty	lru create("c", 5)	Empty LRU, capacity 5	Construction
Push one	push("c", "k1", "v1")	Size 1, get("k1") = "v1"	Basic insert
Push to capacity	Push 5 items	Size 5, all retrievable	Capacity fill
Push beyond capacity	Push 6th item	Size 5, oldest evicted	Eviction
Get updates access	get("k1"), then push 5 new	"k1" survives (recently accessed)	LRU ordering
Peek doesn't update	peek(3), then push to capacity	Peeked items evicted normally	Peek semantics
Contains hit	contains("k1") after push	true	Membership
Contains miss	contains("nonexistent")	false	Membership
Evict specific	evict("k1")	Size decreases, get("k1") fails	Manual eviction
Clear	clear("c")	Size 0	Bulk clear
Duplicate key push	push("k1", "v2") after push("k1", "v1")	get("k1") = "v2", size unchanged	Update semantics
Capacity 1	Create with capacity 1, push twice	Only second item present	Minimum capacity
Large capacity	Create with capacity 1000, fill	All 1000 retrievable	Scale
Deterministic eviction	Same push sequence twice	Same eviction order	Determinism (F1)
Snapshot roundtrip	Snapshot, modify, restore	Original state exactly	Snapshot (F2)
Clone isolation	Clone, modify clone, check source	Source unchanged	Isolation (F4)

Test	Input	Expected	Tests Property
Inherit from parent	Parent has LRU, query from child	Parent's entries visible	Inheritance
Shadow parent	Parent and child both have same-name LRU	Child's entries returned	Shadowing
Empty peek	peek(5) on empty LRU	Empty list	Edge case
Zero-length peek	peek(0)	Empty list	Edge case
Peek more than size	peek(10) on LRU with 3 items	3 items	Edge case
Push Term types	Push fraction, atom, list as values	All retrievable with correct types	Type flexibility
Mutation logged	Push, check mutation log	Log entry exists with correct fields	Provenance
Capacity constraint	Verify size never exceeds capacity	size $\leq$ capacity always	Growth bound (F7)

S.2 Counter Tests (18 tests)

Test	Input	Expected	Tests Property
Create default	counter create("c")	Value 0, min 0, max $2^{31}-1$	Construction
Create bounded	counter create("c", min=-10, max=10)	Value -10, bounds set	Bounded construction
Inc	counter inc("c")	Value 1	Basic increment
Dec	counter dec("c") from 5	Value 4	Basic decrement
Add positive	counter add("c", 10)	Value increased by 10	Delta add
Add negative	counter add("c", -3)	Value decreased by 3	Negative delta

Test	Input	Expected	Tests Property
Clamp at max	Inc past max value	Value stays at max	Upper bound
Clamp at min	Dec past min value	Value stays at min	Lower bound
Reset	counter reset("c") from 42	Value = min value	Reset
Set	counter set("c", 7)	Value = 7	Direct set
Set out of range	counter set("c", max+1)	Clamped or error	Bounds enforcement
Get	counter get("c")	Current value	Read
Deterministic	Same inc/dec sequence twice	Same final value	Determinism (F1)
Snapshot roundtrip	Snapshot, modify, restore	Original value exactly	Snapshot (F2)
Clone isolation	Clone, inc in clone, check source	Source unchanged	Isolation (F4)
Inherit from parent	Parent has counter, get from child	Parent's value	Inheritance
Mutation logged	Inc, check log	Log entry with old/new values	Provenance
Overflow safety	Inc 2^{31} times starting from 0	Clamps at max, no wraparound	Safety

S.3 Lock Tests (15 tests)

Test	Input	Expected	Tests Property
Create	lock create("l")	Free state	Construction
Acquire free	lock acquire("l")	Returns true, now held	Basic acquire
Acquire held	lock acquire("l") on held lock	Returns false, still held by original	Non-blocking

Test	Input	Expected	Tests Property
Release	lock release("l")	Free state	Basic release
Release free	lock release("l") on free lock	Remains free, no error	Idempotent
Holder tracking	lock acquire("l", holder="task 1")	lock holder returns "task 1"	Holder identity
Force release	lock force release("l") on held	Free state, old holder logged	Admin override
Check held	lock check("l") on held	true	State read
Check free	lock check("l") on free	false	State read
Deterministic	Same ac- quire/release sequence	Same final state	Determinism (F1)
Snapshot roundtrip	Snapshot held lock, restore	Lock held by same holder	Snapshot (F2)
Clone isolation	Clone, release in clone, check source	Source still held	Isolation (F4)
Inherit from parent	Parent has lock, check from child	Parent's state visible	Inheritance
Mutation logged	Acquire, check log	Log entry with holder and success	Provenance
Timeout constraint	Lock held for 100+ turns with timeout constraint	Constraint fires	Constraint integration

S.4 Queue Tests (21 tests)

Test	Input	Expected	Tests Property
Create	queue create("q", 5)	Empty, capacity 5	Construction

Test	Input	Expected	Tests Property
Push one	queue push("q", "item1")	Size 1, returns true	Basic push
Pop one	Push then pop	Returns "item1", size 0	Basic pop
FIFO order	Push A, B, C; pop three times	Returns A, B, C in order	FIFO semantics
Push to capacity	Push 5 items	Size 5, is full = true	Capacity fill
Push when full	Push 6th item	Returns false, size still 5	Reject on full (F7)
Pop when empty	Pop on empty queue	Returns none	Empty handling
Peek	Push A, B; peek	Returns A, size unchanged	Non-destructive read
Peek empty	Peek on empty	Returns none	Edge case
Is empty true	On empty queue	true	State test
Is empty false	After push	false	State test
Is full true	At capacity	true	State test
Is full false	Below capacity	false	State test
Clear	Clear queue with items	Size 0, is empty = true	Bulk clear
To list	Push A, B, C; to list	[A, B, C] in FIFO order	Read all
Capacity 1	Create capacity 1, push, push	Second push returns false	Minimum capacity
Deterministic	Same push/pop sequence twice	Same final state	Determinism (F1)
Snapshot roundtrip	Snapshot, modify, restore	Original order exactly	Snapshot (F2)
Clone isolation	Clone, pop in clone, check source	Source unchanged	Isolation (F4)
Mutation logged	Push, check log	Log entry with item and new size	Provenance

Test	Input	Expected	Tests Property
Term types	Push fraction, atom, list	All pop correctly typed	Type flexibility

S.5 Stack Tests (18 tests)

Test	Input	Expected	Tests Property
Create	stack	Empty, capacity 5	Construction
Push one	create("s", 5) stack push("s", "item1")	Size 1, returns true	Basic push
Pop one	Push then pop	Returns "item1", size 0	Basic pop
LIFO order	Push A, B, C; pop three times	Returns C, B, A in order	LIFO semantics
Push to capacity	Push 5 items	Size 5	Capacity fill
Push when full	Push 6th item	Returns false, size still 5	Reject on full (F7)
Pop when empty	Pop on empty stack	Returns none	Empty handling
Peek	Push A, B; peek	Returns B, size unchanged	Non-destructive read
Is empty	On empty	true	State test
Clear	Clear stack with items	Size 0	Bulk clear
To list	Push A, B, C; to list	[C, B, A] top-to-bottom	Read all
Capacity 1	Create capacity 1, push, push	Second push returns false	Minimum capacity
Deterministic	Same push/pop sequence	Same final state	Determinism (F1)
Snapshot roundtrip	Snapshot, modify, restore	Original order exactly	Snapshot (F2)
Clone isolation	Clone, pop in clone	Source unchanged	Isolation (F4)
Mutation logged	Push, check log	Log entry present	Provenance
Undo/redo pattern	Push states, pop to undo	Previous state recovered	Use case

Test	Input	Expected	Tests Property
Mixed types	Push different Term types	All pop correctly	Type flexibility

S.6 Ring Buffer Tests (15 tests)

Test	Input	Expected	Tests Property
Create	ring create("r", 5)	Empty, capacity 5	Construction
Write one	ring write("r", "item1")	Size 1	Basic write
Write to capacity	Write 5 items	Size 5	Capacity fill
Write beyond capacity	Write 6th item	Size 5, oldest overwritten	Circular overwrite
Read all in order	Write A, B, C; read all	[A, B, C] chronological	Order
Read all after overwrite	Write 7 items to capacity-5 buffer	Last 5 items chronological	Circular read
Read last N	Write 5 items; read last(3)	Last 3 items	Partial read
Read last more than size	Write 3 items; read last(5)	3 items	Edge case
Clear	Clear with items	Size 0	Bulk clear
Size tracks correctly	Write and clear alternately	Size always accurate	Count tracking
Deterministic	Same write sequence	Same contents and order	Determinism (F1)
Snapshot roundtrip	Snapshot, write more, restore	Original contents and write pos	Snapshot (F2)
Clone isolation	Clone, write in clone	Source unchanged	Isolation (F4)
Mutation logged	Write, check log	Log entry with item and overwrite info	Provenance
Capacity 1	Create capacity 1, write twice	Only second item present	Minimum capacity

S.7 Bitset Tests (18 tests)

Test	Input	Expected	Tests Property
Create	bitset create("b", 100)	All clear, width 100	Construction
Set bit	bitset set("b", 0)	test(0) = true	Basic set
Clear bit	Set then clear bit 0	test(0) = false	Basic clear
Test unset	test(50) on fresh bitset	false	Default state
Count zero	Count on fresh bitset	0	Empty count
Count after sets	Set bits 0, 5, 10; count	3	Populated count
All set false	After setting some bits	false	Partial fill
All set true	Set all bits	true	Full fill
Any set false	Fresh bitset	false	Empty test
Any set true	After setting one bit	true	Single bit
Reset	Set bits, then reset	All clear, count = 0	Bulk clear
To list	Set bits 3, 7, 11; to list	[3, 7, 11]	Index extraction
Out of range	Set bit at index = width	Error	Bounds (F7)
Width 1	Create width 1, set, test	Works correctly	Minimum width
Deterministic	Same set/clear sequence	Same final bits	Determinism (F1)
Snapshot roundtrip	Snapshot, modify, restore	Original bits exactly	Snapshot (F2)
Clone isolation	Clone, set in clone	Source unchanged	Isolation (F4)
Mutation logged	Set bit, check log	Log entry with index and was already set	Provenance

S.8 Test Count Summary

Primitive	Tests	Edge Cases	Invariant Tests	Integration Tests
LRU	24	5	6	5
Counter	18	3	5	4
Lock	15	2	5	3
Queue	21	4	5	4
Stack	18	3	5	3
Ring Buffer	15	3	4	3
Bitset	18	3	5	3
Total	129	23	35	25

Appendix T: Cross-Paper Traceability Matrix

T.1 VDR-8 Features Traced to Prior Paper Foundations

VDR-8 Feature	Depends On	From Paper	Specific Section
Data primitives in KB struct	KB struct definition	VDR-5	§11.4 KnowledgeBase struct
Data primitives in KB struct	Constraints-in-KB pattern	VDR-5 Addendum	§A1.2 Revised KB Structure
Scoped data primitives	KB scope resolution	VDR-5	§5.1 The Scoping Principle
Data primitive inheritance	Working data inheritance	VDR-5	§6.2 Inheritance and Shadowing
Command token access	Command token mechanism	VDR-6	§5 Command Tokens
Pure primitive classification	Pure primitive contract	VDR-6	§2 Pure Primitives
Non-blocking lock design	Turn-based execution model	VDR-6	§7 Async Task Management
Dotted-path naming	KB tree hierarchy	VDR-5	§5.2 The KB Tree
Dotted-path naming	Lifecycle KB tree	VDR-7	§14 Cross-Phase KB Relationships
Integer ID acceleration	i32 preference	User preferences	Zig 0.14, prefer i32
Mount system	KB activation/deactivation	VDR-5	§5.5 KB Activation
Connections field	Self-describing KB principle	VDR-5 Addendum	§A3.1 Self-Describing Data

VDR-8 Feature	Depends On	From Paper	Specific Section
Session snapshot	Checkpoint KB pattern	VDR-7	§6.4 Checkpoint KB
Session clone	Version branching	VDR-7	§12.2 Update Types
Disposable clones	Canary deployment pattern	VDR-7	§12.5 Canary Deployment
Drift detection	Constraint-based monitoring	VDR-7	§11 Monitoring
Drift detection	Watch system	VDR-6 Appendix J	Reminder and Watch Reference
Mutation logging	Provenance recording	VDR-5	§4.2 Facts (asserted at, derivation)
Reference-based commands	Root KB always in scope	VDR-5	§5.1 Global KB always searched
Scratchpad as ring buffer	Scratchpad concept	VDR-6	§5.4 Scratchpad Channel

T.2 VDR-8 Features That Enrich Prior Specifications

Prior Specification	VDR-8 Enrichment	How
VDR-5 KB struct	8 new live state fields + 3 structural fields	Data primitives, connections, mounts, path, id
VDR-5 scope resolution	Mount-aware scope walking	Mounts expand scope to cross-branch KBs
VDR-5 working data	Reclassified as live state	Clear lifecycle: live vs persistent
VDR-6 scratchpad	Formalized as ring buffer	Named, bounded, specified

Prior Specification	VDR-8 Enrichment	How
VDR-6 command tokens	Path-based argument references	Arguments by address, not by value
VDR-6 command tokens	Lower generative burden	Reference selection vs text generation
VDR-6 primitive count	+78 new pure primitives	255 → 333 total
VDR-7 KB naming	Structured dotted-path namespace	Hierarchical addressing replaces flat names
VDR-7 checkpoint pattern	Generalized to session snapshots	Any live state capturable, not just model weights
VDR-7 canary pattern	Generalized to disposable clones	Any session recyclable, not just deployments
VDR-7 lifecycle KB tree	Navigable via connections graph	Explicit typed links replace implicit parent-child

END VDR-8 EXTENDED APPENDIX TABLES

[[HOWL-VDR-8-2026](#)] Computational State Primitives, Universal Data Pathing, and

Session Management. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-8-2026>

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-3-2026](#)] VDR Gym Extension. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-3-2026>

[[HOWL-VDR-4-2026](#)] Exact-Fraction Language Model Architecture. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-4-2026>

[[HOWL-LLM-1-2026](#)] Integer LLM with Prolog Knowledge Base. Github: <https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-1-2026>

[[HOWL-VDR-5-2026](#)] Exact Arithmetic Meets Logical Provenance. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-5-2026>

[[HOWL-VDR-6-2026](#)] Computational Primitives and Operational Environments. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-6-2026>

[[HOWL-VDR-7-2026](#)] Complete Lifecycle Technical Specification. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-7-2026>